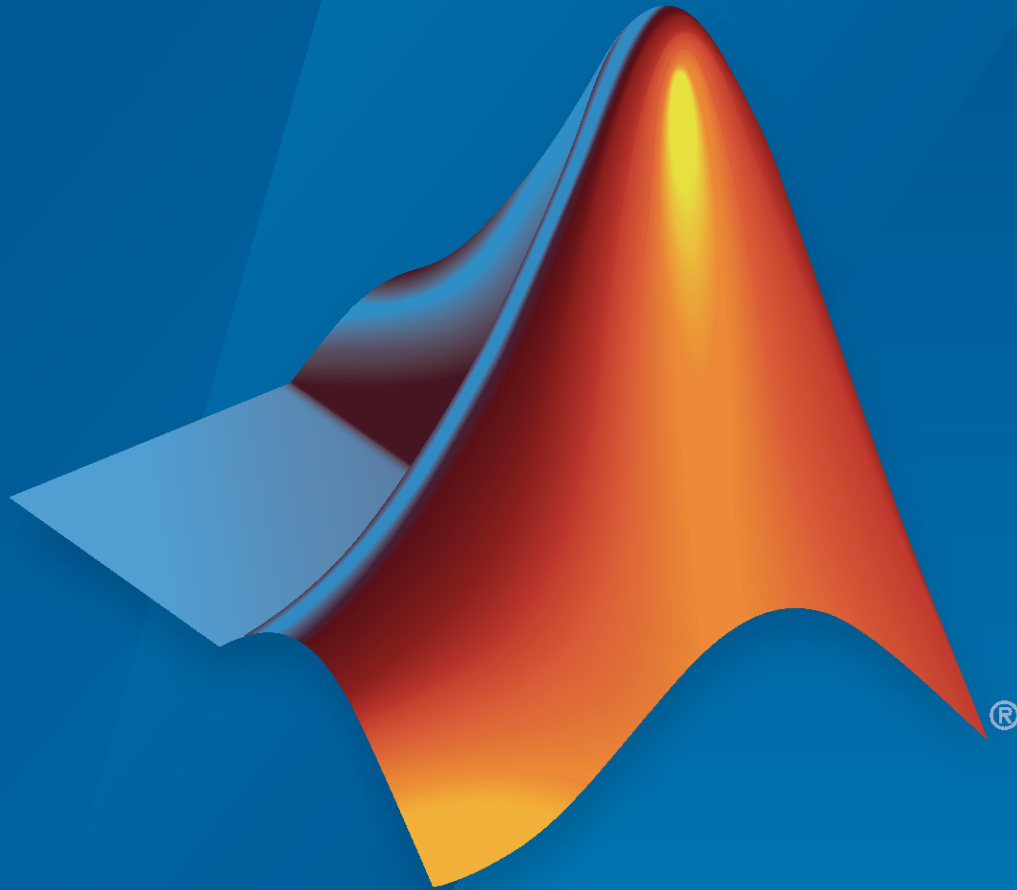


Stateflow®

Getting Started Guide



MATLAB® & SIMULINK®

R2026a



How to Contact MathWorks



Latest news: www.mathworks.com
Sales and services: www.mathworks.com/sales_and_services
User community: www.mathworks.com/matlabcentral
Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Stateflow® Getting Started Guide

© COPYRIGHT 2004-2026 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

June 2004	First printing	New for Version 6.0 (Release 14)
October 2004	Online only	Revised for Version 6.1 (Release 14SP1)
March 2005	Online only	Revised for Version 6.2 (Release 14SP2)
September 2005	Online only	Revised for Version 6.3 (Release 14SP3)
October 2005	Reprint	Version 6.0
March 2006	Second printing	Revised for Version 6.4 (Release 2006a)
September 2006	Reprint	Version 6.5 (Release 2006b)
March 2007	Online only	Rereleased for Version 6.6 (Release 2007a)
September 2007	Third printing	Rereleased for Version 7.0 (Release 2007b)
March 2008	Fourth printing	Revised for Version 7.1 (Release 2008a)
October 2008	Fifth printing	Revised for Version 7.2 (Release 2008b)
March 2009	Sixth printing	Revised for Version 7.3 (Release 2009a)
September 2009	Online only	Revised for Version 7.4 (Release 2009b)
March 2010	Online only	Revised for Version 7.5 (Release 2010a)
September 2010	Online only	Revised for Version 7.6 (Release 2010b)
April 2011	Seventh printing	Revised for Version 7.7 (Release 2011a)
September 2011	Online only	Revised for Version 7.8 (Release 2011b)
March 2012	Online only	Revised for Version 7.9 (Release 2012a)
September 2012	Online only	Revised for Version 8.0 (Release 2012b)
March 2013	Online only	Revised for Version 8.1 (Release 2013a)
September 2013	Online only	Revised for Version 8.2 (Release 2013b)
March 2014	Online only	Revised for Version 8.3 (Release 2014a)
October 2014	Online only	Revised for Version 8.4 (Release 2014b)
March 2015	Online only	Revised for Version 8.5 (Release 2015a)
September 2015	Online only	Revised for Version 8.6 (Release 2015b)
October 2015	Online only	Rereleased for Version 8.5.1 (Release 2015aSP1)
March 2016	Online only	Revised for Version 8.7 (Release 2016a)
September 2016	Online only	Revised for Version 8.8 (Release 2016b)
March 2017	Online only	Revised for Version 8.9 (Release 2017a)
September 2017	Online only	Revised for Version 9.0 (Release 2017b)
March 2018	Online only	Revised for Version 9.1 (Release 2018a)
September 2018	Online only	Revised for Version 9.2 (Release 2018b)
March 2019	Online only	Revised for Version 10.0 (Release 2019a)
September 2019	Online only	Revised for Version 10.1 (Release 2019b)
March 2020	Online only	Revised for Version 10.2 (Release 2020a)
September 2020	Online only	Revised for Version 10.3 (Release 2020b)
March 2021	Online only	Revised for Version 10.4 (Release 2021a)
September 2021	Online only	Revised for Version 10.5 (Release 2021b)
March 2022	Online only	Revised for Version 10.6 (Release 2022a)
September 2022	Online only	Revised for Version 10.7 (Release 2022b)
March 2023	Online only	Revised for Version 10.8 (Release 2023a)
September 2023	Online only	Revised for Version 23.2 (R2023b)
March 2024	Online only	Revised for Version 24.1 (R2024a)
September 2024	Online only	Revised for Version 24.2 (R2024b)
March 2025	Online only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Revised for Version 25.2 (R2025b)
March 2026	Online only	Revised for Version 26.1 (R2026a)

1	Introduction to the Stateflow Product	
	Stateflow Product Description	1-2

2	Stateflow Fundamentals	
	Design Finite State Machines in Stateflow	2-2
	Define Logic by Using States and Transitions	2-2
	Model State Machines by Using State Transition Tables and Charts	2-2
	Model Rechargeable Battery System as Chart	2-5
	Model Battery System	2-5
	Model Rechargeable Battery System by Using a State Transition Table	2-7
	Create Stateflow Charts	2-10
	Create Chart	2-10
	Add States	2-11
	Connect States	2-12
	Add Executable Code	2-15
	Define Chart Data and Share with Simulink Model	2-15
	Connect Simulink Blocks to Chart	2-17
	Simulate Model	2-18
	Log, Verify, and Debug Charts	2-19
	Open Model	2-19
	Log States and Data	2-19
	Set Breakpoints	2-21
	Create Parent and Child Operating Modes	2-23
	Connect Transitions by Using Junctions	2-29
	Execute States in Parallel	2-37
	Reuse Logic in Charts	2-43
	Open Model	2-43
	Create MATLAB Function	2-45
	Add MATLAB Function to State Actions	2-47
	Simulate Model	2-48

Create State Transition Tables	2-50
Create Table	2-50
Add States	2-51
Select Initial Active State	2-52
Move Between States	2-52
Add Executable Code	2-54
Define Table Data and Share with Simulink Model	2-55
Connect Simulink Blocks to Table	2-57
Simulate Model	2-57
Log, Verify, and Debug State Transition Tables	2-59
Open Model	2-59
Log States and Data	2-60
Set Breakpoints	2-61
Create Parent and Child Operating Modes in State Transition Tables ..	2-63
Open Model	2-63
Add Child States to Model Hierarchy	2-64
Simulate Model	2-67
Execute States in Parallel in State Transition Tables	2-69
Open Model	2-69
Enable Parallel Decomposition	2-71
Coordinate Behavior Across Parallel States	2-73
Simulate Model	2-75

Introduction to the Stateflow Product

Stateflow Product Description

Model and simulate decision logic using state machines and flow charts

Stateflow is a product that provides a graphical language that includes state transition diagrams, flow charts, state transition tables, and truth tables. You can use Stateflow to describe how MATLAB[®] algorithms and Simulink[®] models react to input signals, events, and time-based conditions.

Stateflow enables you to design and develop supervisory control, task scheduling, fault management, communication protocols, user interfaces, and hybrid systems.

With Stateflow, you model combinatorial and sequential decision logic that can be simulated as a block within a Simulink model or executed as an object in MATLAB. Graphical animation enables you to analyze and debug your logic while it is executing. Edit-time and run-time checks ensure design consistency and completeness before implementation.

Stateflow Fundamentals

This chapter describes the fundamental concepts of event-based modeling in Stateflow.

- “Design Finite State Machines in Stateflow” on page 2-2
- “Model Rechargeable Battery System as Chart” on page 2-5
- “Model Rechargeable Battery System by Using a State Transition Table” on page 2-7
- “Create Stateflow Charts” on page 2-10
- “Log, Verify, and Debug Charts” on page 2-19
- “Create Parent and Child Operating Modes” on page 2-23
- “Connect Transitions by Using Junctions” on page 2-29
- “Execute States in Parallel” on page 2-37
- “Reuse Logic in Charts” on page 2-43
- “Create State Transition Tables” on page 2-50
- “Log, Verify, and Debug State Transition Tables” on page 2-59
- “Create Parent and Child Operating Modes in State Transition Tables” on page 2-63
- “Execute States in Parallel in State Transition Tables” on page 2-69

Design Finite State Machines in Stateflow

Stateflow is a graphical programming environment that allows you to use finite state machines to create control logic in Simulink models. Additionally, you can:

- Reuse and share your logic across models.
- Use MATLAB code or Simulink subsystems as functions in a state machine.
- Visualize your logic with live animation.
- Debug your logic by using breakpoints and logging simulation data.

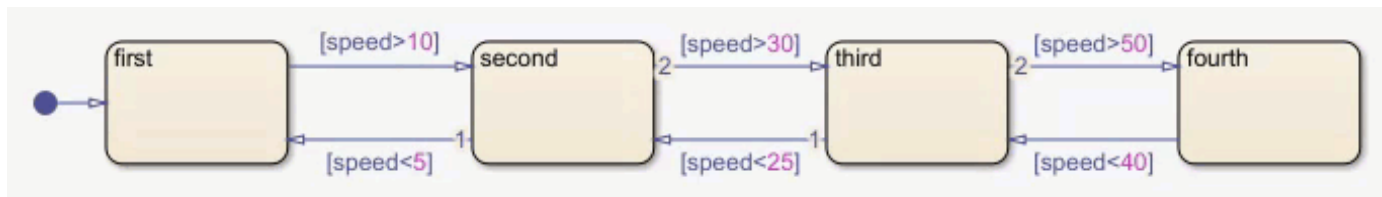
Stateflow also supports truth tables and flow charts. For more information, see “Use Truth Tables to Model Combinatorial Logic” and “Create Flow Charts in Stateflow”.

Define Logic by Using States and Transitions

Finite state machines model the operating modes of a system as *states*. A state can modify data, execute code and events, or contain other states.

Finite state machines execute in a series of steps. During each step, each state is either active or inactive. Active states can perform actions or execute code, while inactive states cannot. As the conditions in the state machine change, so do the active states.

Consider an automatic transmission in a car, in which the gears are the operating mode. This animated image models the gears as states, represented by rectangles. As the speed of the car changes, the transmission shifts gears, represented by blue highlighting.



In a state machine, a transition represents the path from one state to another. For example, an automatic transmission system can shift from first gear to second gear, from second gear to first or third gear, and so on.

The animated image represents the transitions in the transmission system as arrows between states. Each transition has a condition that determines when the active state changes. For example, the car shifts from first to second gear when the speed of the car becomes greater than 10.

Model State Machines by Using State Transition Tables and Charts

With Stateflow, you can model state machines as State Transition Table or Chart blocks.

Use State Transition Tables

State transition tables represent states and transitions as entries in a table. For example, this state transition table models the gears of an automatic transmission as states.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
	[speed>10]	
	second ▼	▼
second	[speed<5]	[speed>30]
	first ▼	third ▼
third	[speed<25]	[speed>50]
	second ▼	fourth ▼
fourth	[speed<40]	
	third ▼	▼

Consider using a state transition table if you want to focus on implementing logic without worrying about the graphical layout of your states.

For a tutorial on creating state transition tables, see “Model Rechargeable Battery System by Using a State Transition Table” on page 2-7.

Use Charts

When you create a chart, you model the state machine by arranging rectangles and arrows that represent states and transitions on a canvas, as in the previous animation. Consider using a chart if your design requires:

- Reusable components. For example, you can reuse states in multiple places in one chart, or use the same state in multiple charts.
- Event-driven operating modes. For example, you can change states when one component of a chart receives a signal from a different component.
- Non-linear flow. For example, you can design a logical flow that uses loops or branching paths.

For a tutorial on creating Stateflow charts, see “Model Rechargeable Battery System as Chart” on page 2-5.

See Also

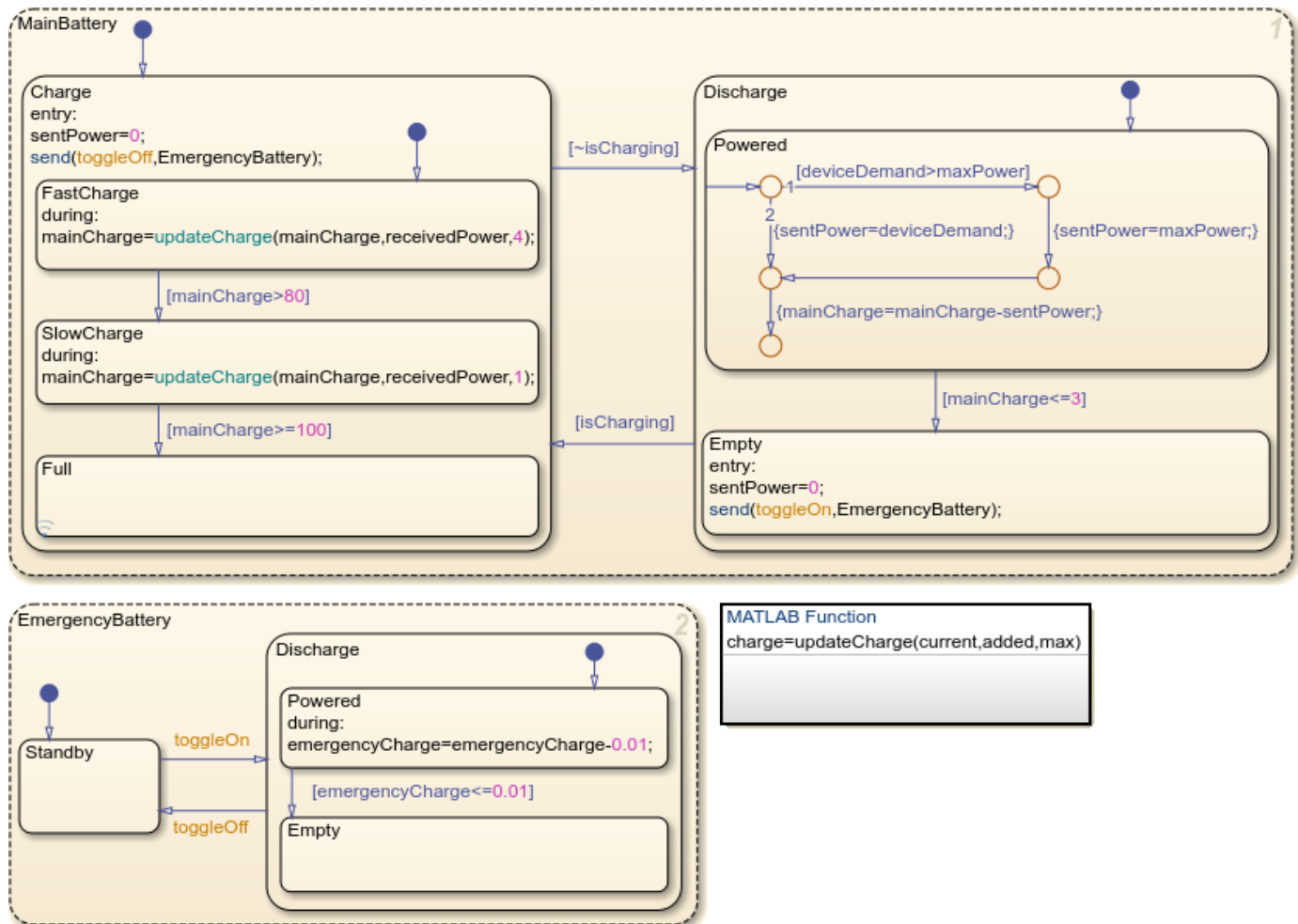
Related Examples

- “Stateflow Semantics”
- “Model a Finite State Machine”
- “Model Finite State Machines Using State Transition Tables”
- “Overview of Mealy and Moore Machines”

Model Rechargeable Battery System as Chart

Stateflow charts use a graphical canvas to model the elements of a finite state machine. For more information about finite state machines, see “Design Finite State Machines in Stateflow” on page 2-2.

In this tutorial, you create a Stateflow chart that models a rechargeable battery system. The system has two batteries that contain multiple operating modes, share reusable components, and execute in parallel. The Stateflow chart exists as a block inside a Simulink model, and inputs data from and outputs data to other blocks in the model.



Model Battery System

In the tutorial, you start with an empty chart, and in each topic, you add new functionality. The tutorial has these topics.

- 1 “Create Stateflow Charts” on page 2-10: Create a chart executes code and shares data.
- 2 “Log, Verify, and Debug Charts” on page 2-19: Use debugging tools to understand how a chart executes and fix unintended behavior.
- 3 “Create Parent and Child Operating Modes” on page 2-23: Create a hierarchy of nested states.

- 4 “Connect Transitions by Using Junctions” on page 2-29: Create multi-step and branching paths between states.
- 5 “Execute States in Parallel” on page 2-37: Activate multiple states in the same time step.
- 6 “Reuse Logic in Charts” on page 2-43: Use functions to create reusable components.

See Also

Related Examples

- “Stateflow Semantics”
- “Model a Finite State Machine”
- “Overview of Mealy and Moore Machines”

Model Rechargeable Battery System by Using a State Transition Table

State transition tables model finite state machine machines as tables, where the rows represent states and the columns represent transitions. For more information about finite state machines, see “Design Finite State Machines in Stateflow” on page 2-2.

In this tutorial, you create a state transition table that models a rechargeable battery system. The system has two batteries that contain multiple operating modes and execute in parallel. The state transition table is a block in a Simulink model, and receives data from and outputs data to other blocks in the model.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
Main		
Charge entry: sentPower=0; emerOn=0;	[~isCharging]	
FastCharge during: mainChrg=mainChrg+3;		
SlowCharge during: mainChrg=mainChrg+1;		
Full		
Discharge entry: sentPower=3.5; during:		

In this tutorial, you start with a basic state transition table and add new functionality.

- 1 “Create State Transition Tables” on page 2-50: Create a state transition table that executes code and shares data.
- 2 “Log, Verify, and Debug State Transition Tables” on page 2-59: Use debugging tools to understand how a table executes.
- 3 “Create Parent and Child Operating Modes in State Transition Tables” on page 2-63: Create a hierarchy of nested states.
- 4 “Execute States in Parallel in State Transition Tables” on page 2-69: Activate multiple states in the same time step.

See Also

Related Examples

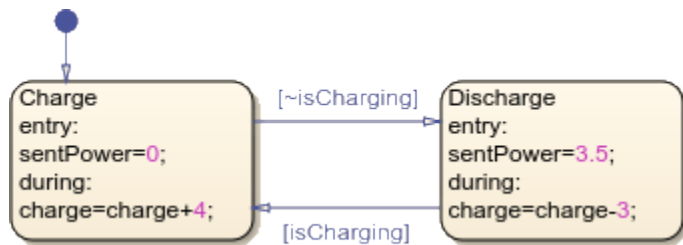
- [“Stateflow Semantics”](#)
- [“Model Finite State Machines Using State Transition Tables”](#)
- [“Overview of Mealy and Moore Machines”](#)

Create Stateflow Charts

In this tutorial, you use a Stateflow chart to model the logic of a rechargeable battery system.

The battery system has these requirements:

- The battery charges when connected to an external power source. Otherwise, it discharges.
- The battery capacity charges at a rate of 4% of total charge and discharges rate of 3%.
- When charging, the battery does not output power. When discharging, the battery outputs 3.5 watts of power.

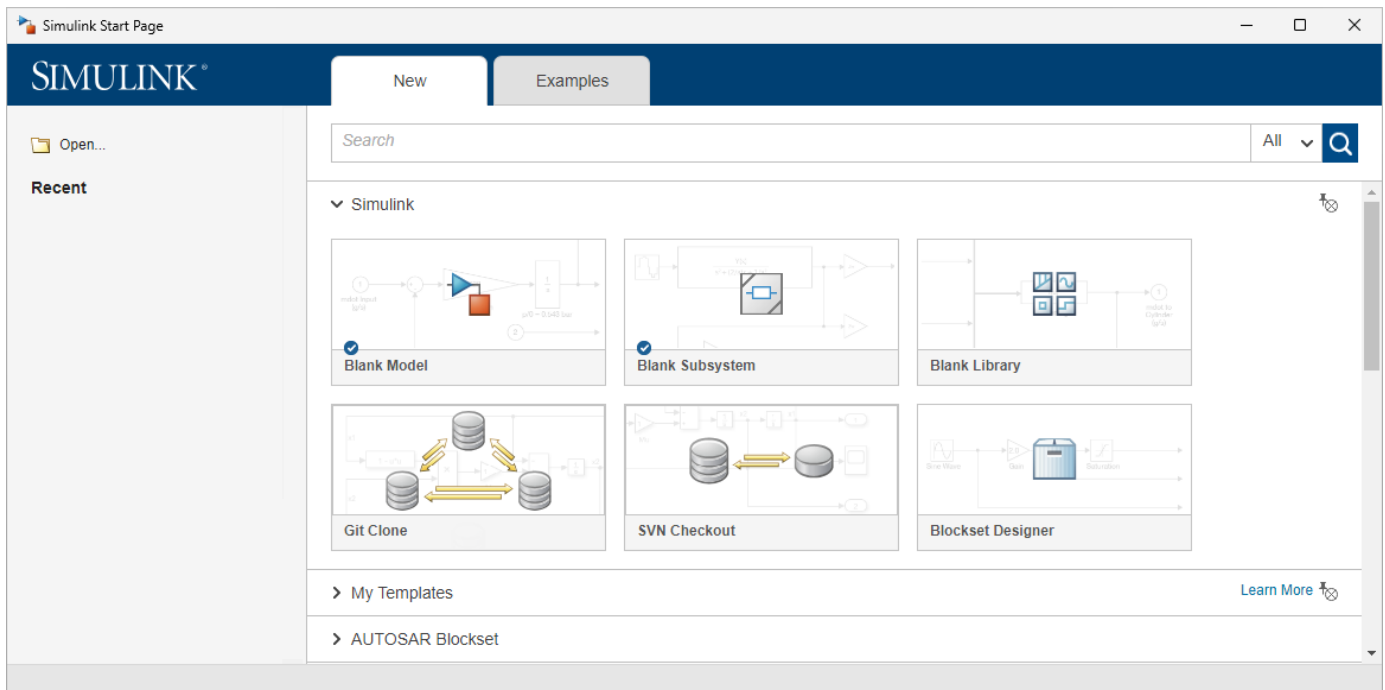


To model these requirements, you build a chart that contains two states, Charge and Discharge, that represent the operating modes of the battery system.

Create Chart

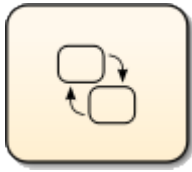
Create a new Simulink model that contains an empty Chart block.

- 1 Start MATLAB. In the MATLAB Toolstrip, in the **Home** tab, click **Simulink**.



- 2 On the start page, in the **Stateflow** section, click the **Blank Chart** template.

The Simulink Editor opens and displays a model that contains a Chart block.



- 3 Open the Stateflow Editor by double-clicking the Chart block.

The Stateflow Editor has these components:

- **Canvas** — The graphical environment in which you place states and transitions. The canvas has a cream-colored background by default.
- **Explorer Bar** — A rectangular area, located above the canvas, that displays the path to the open chart or graphical element. You can move between the Stateflow and Simulink Editors by clicking the arrow buttons or model elements.
- **Palette** — A menu, located to the left of the canvas, from which you add objects to the canvas. It contains icons for states  and other chart elements.

To display the object names, right-click the palette and click **Show Names**. To hide the names, right-click and select **Hide Names**.

- **Symbols** pane — A pane, located to the right of the canvas by default, in which you create and manage data, events, and messages that allow the chart to communicate with the rest of the Simulink model.

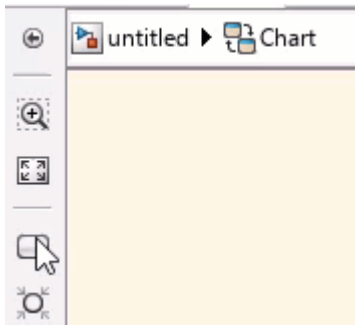
To open or close the **Symbols** pane, in the **Modeling** tab, click **Symbols Pane**.



Add States

The battery system requires two states, one for charging and one for discharging. Use the palette to add the two states to the canvas.

- 1 In the palette, click the state icon . To place the state, point to a blank section of the canvas and click.



- 2 When you place a state, the editor prompts you to enter a state name in the upper-left corner of the state. Enter **Charge**.

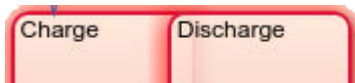
To finalize the state name, click on a blank section of the canvas. To edit an existing state name, click the text inside the state.

Note State names cannot contain spaces or begin with a number. Each state name must be unique.

- 3 Add a second state and name it **Discharge**.



Note State borders must not overlap.



Connect States

Transitions determine how and when your chart moves between states.

Use Default Transitions to Indicate First Active State

A blue circle  indicates a default transition, which determines which state becomes active when the simulation starts.

The chart places a default transition on the first state you add to the canvas. In this example, the default transition connects to the **Charge** state. You can add other default transitions from the palette by clicking the default transition icon  and clicking the edge of a state.

Because the requirements state that the battery must start in the charging mode, you do not need to move the default transition.

Add Transitions Between States

Transitions between states allow the chart to move from one state to another.

- 1 To create the first transition, point to the edge of the Charge state until your cursor turns into a plus symbol . Click and drag to the edge of the Discharge state.



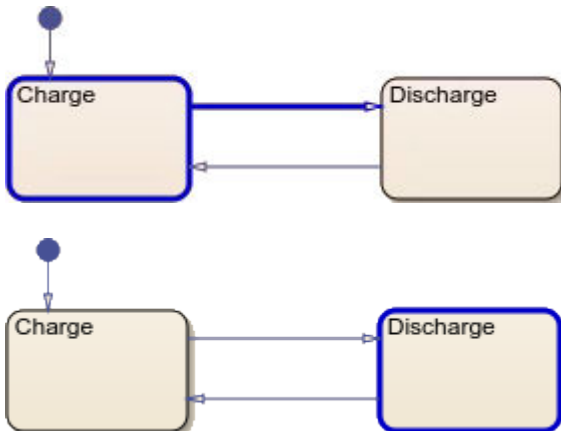
Tip To move an existing transition, click the arrowhead and drag.

- 2 Create a transition from Discharge to Charge.



- 3 Simulate the model. In the **Simulation** tab, click **Run**.

When a state becomes active, the chart highlights the borders of the state. When the chart moves along a transition, it briefly highlights the transition. During simulation, the chart switches between the Charge and Discharge states at each step.



Tip To change the animation speed, in the **Debug** tab, click the **Animation Speed** drop-down menu and select an option.

Add Transition Labels

To change the behavior of transitions, you can add transition labels. Transition labels have three optional components:

- An event trigger or message trigger that prevents the chart from moving along the transition until the transition receives an event or message broadcast from another object in the chart or model.
- A *condition* that must be met before the chart can move along the transition. To specify a condition, use square brackets.

Tip When a chart evaluates a transition without a condition, the chart always moves along the transition.

- An *action* that executes when the chart moves along the transition. To specify an action, use curly braces.

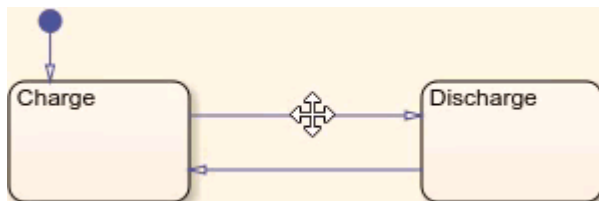


When you first create a transition, the Editor prompts you to enter a label. Alternatively, you can add a transition label by double-clicking the transition.

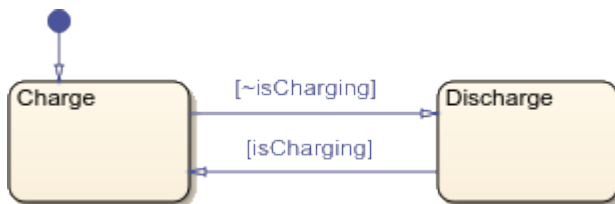
Add conditions to the transitions between the states.

- 1 To transition from Charge to Discharge only when the battery is not charging, double-click the transition and enter the label `[~isCharging]`.

To finalize the label, click the canvas. To move the label, click the center of the label text and drag.



- 2 To transition from Discharge to Charge only when the battery is charging, enter the label `[isCharging]`.



Add Executable Code

You can execute code in active states by adding state actions in the state label. State actions contain a keyword, followed by a colon and a block of executable code.

In this example, you use three types of state actions.

State Action	Behavior
entry	Executes when the state becomes active.
during	Executes every step a state is active. Does not execute on a step when the state becomes active or becomes inactive.
exit	Executes when the state becomes inactive.

Add state actions that change the battery output and charge according to the operating mode.

- 1 In the Charge state, edit the state label by clicking the state name. Add a new line, then enter the text below. You can add new lines by pressing **Enter**.

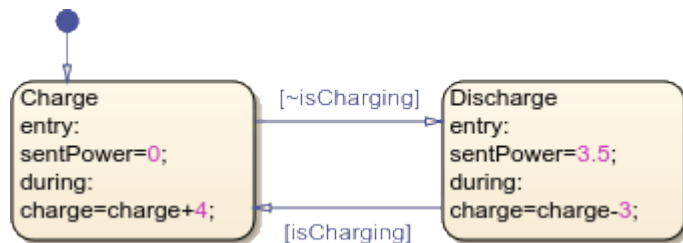
```
entry:
sentPower=0;
during:
charge=charge+4;
```

The entry action sets a variable named `sentPower` to 0. The during action increments a variable named `charge` by 4.

Tip To manually resize a state, click any corner and drag. To automatically reformat every object on the canvas, deselect any selected objects by clicking a blank section of canvas. Then, press **Ctrl+Shift+A**.



- 2 In the Discharge state, add an entry action that sets `sentPower` to 3.5 and a during action that decrements `charge` by 3.

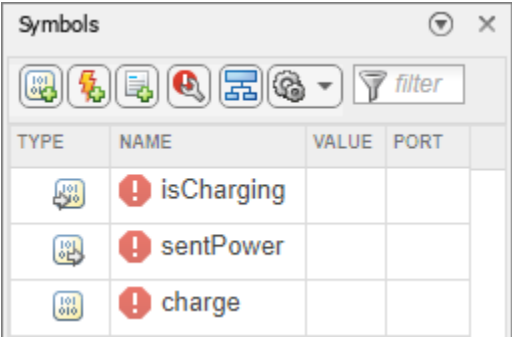


Define Chart Data and Share with Simulink Model

When you use a variable in a transition or state, you must define the variable as input data, output data, or local data. In the **Symbols** pane, the warning badge  indicates undefined data.

Icon	Type	Behavior
	Input Data	During simulation, this data receives the value of the input signal. Defining an input data adds an input port to the Chart block in Simulink. You cannot manually assign values to input data.
	Output Data	During simulation, the chart outputs the value of this data to Simulink. Defining an output data adds an output port to the Chart block in Simulink.
	Local Data	During simulation, this data stores information that is only accessible in the chart.

The chart infers the type of each data based on context. For example, the chart infers that `isCharging` is an input data, `sentPower` is an output data, and `charge` is a local data.



Symbols			
			
TYPE	NAME	VALUE	PORT
	 <code>isCharging</code>		
	 <code>sentPower</code>		
	 <code>charge</code>		

Define the type and value of the chart data.

- 1 To accept the inferred data types, in the **Symbols** pane, click the Resolve undefined symbols button . The warning badges next to the undefined data disappear.
- 2 Set the initial charge of the battery. In the **Symbols** pane, the **Value** column specifies the initial value of each data during simulation. Data with an undefined value defaults to 0.

In the `charge` row, click the **Value** column and enter 50.

Symbols			
TYPE	NAME	VALUE	PORT
	isCharging		1
	sentPower		1
	charge	50	

During simulation, the **Value** column updates as the value of each data changes. When the simulation ends, the **Value** column returns to the original assigned value. You cannot edit data in the Symbols pane during simulation.

- To return to the top level of the Simulink model, in the explorer bar, click the Up to Parent button .

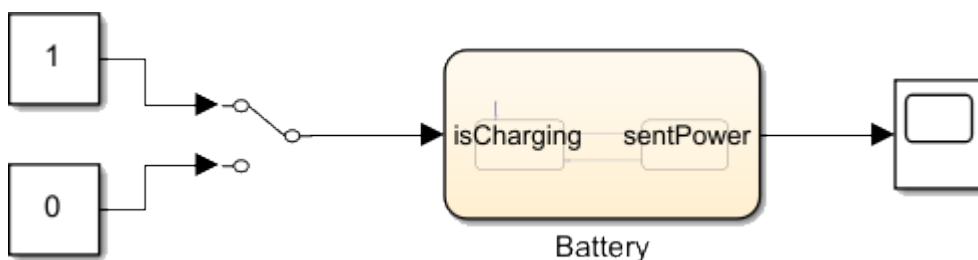
The Chart block has an input and output port. To see the port names, expand the Chart block by clicking a corner and dragging outward.



Connect Simulink Blocks to Chart

To complete the model, connect source and sink blocks to the input and output ports of the Chart block.

- To represent the battery system that connects or disconnects from an external power source, add a Manual Switch block to the Simulink canvas. Connect the output to the input of the Chart block.
- Add a Constant block with a value of 1. Connect the output to the first input port of the Manual Switch block.
- Add a Constant block with a value of 0. Connect the output to the second input port of the Manual Switch block.
- Add a Scope block. Connect the output port of the Chart block to the input port of the Scope block.
- Name the Chart block **Battery**.



Simulate Model

Simulate the completed model.

- 1** In the **Simulation** tab, set **Stop Time** to Inf.
- 2** Double-click to enter the Chart block.
- 3** To simulate the model, in the **Modeling** tab, click **Run**. Observe the blue highlighting around the Charge state.
- 4** Return to the Simulink Editor.
- 5** To toggle the Manual Switch block, double-click the block.
- 6** Open the Stateflow Editor. Observe the blue highlighting around the Discharge state.
- 7** To end the simulation, in the **Modeling** tab, click **Stop**.
- 8** Toggle the Manual Switch to 1.

In the next step of the tutorial, you use active state output, logging, and breakpoints to verify and debug the battery model.

See Also

Related Examples

- “Stateflow Editor Operations”
- “Set Data Properties”
- “Eliminate Redundant Code by Combining State Actions”

Log, Verify, and Debug Charts

In Stateflow, you can log data and states to understand when and how they change during simulation. You can add breakpoints to pause execution on states or transitions. By combining these techniques, you can verify and debug your chart logic.

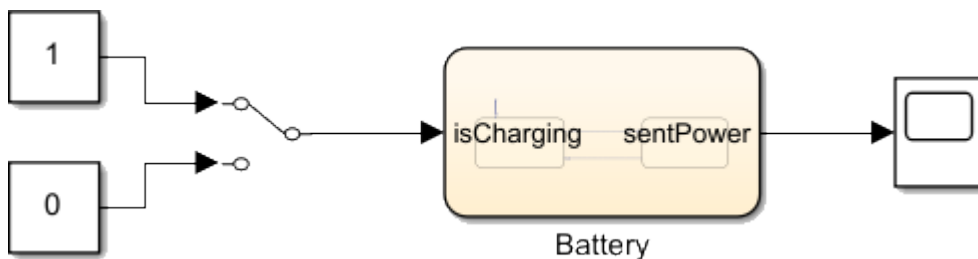
In this example, you identify a design problem in a rechargeable battery model by logging the state activity and data values of a chart. Then, you use breakpoints to identify the state and time step in which the design problem occurs.

Open Model

To build the model, follow the instructions in the previous step of the tutorial. Alternatively, open the `sfGetStartedBattery` model from the MATLAB command prompt.

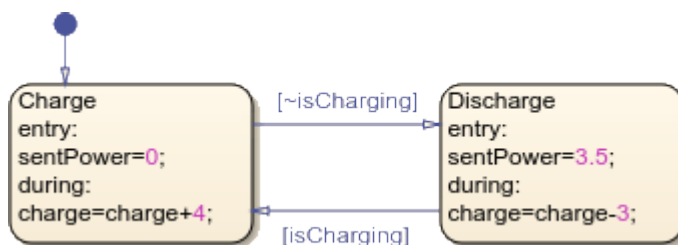
- 1 In the MATLAB command prompt, enter:

```
openExample("sfGetStartedBattery")
```



You can charge or discharge the battery by toggling the Manual Switch block.

- 2 Double-click the Chart block to enter the Stateflow Editor.



The states **Charge** and **Discharge** represent the operating modes of the battery system. The input `isCharging` determines the active state. The data `sentPower` and `charge` represent the output wattage and charge level of the battery.

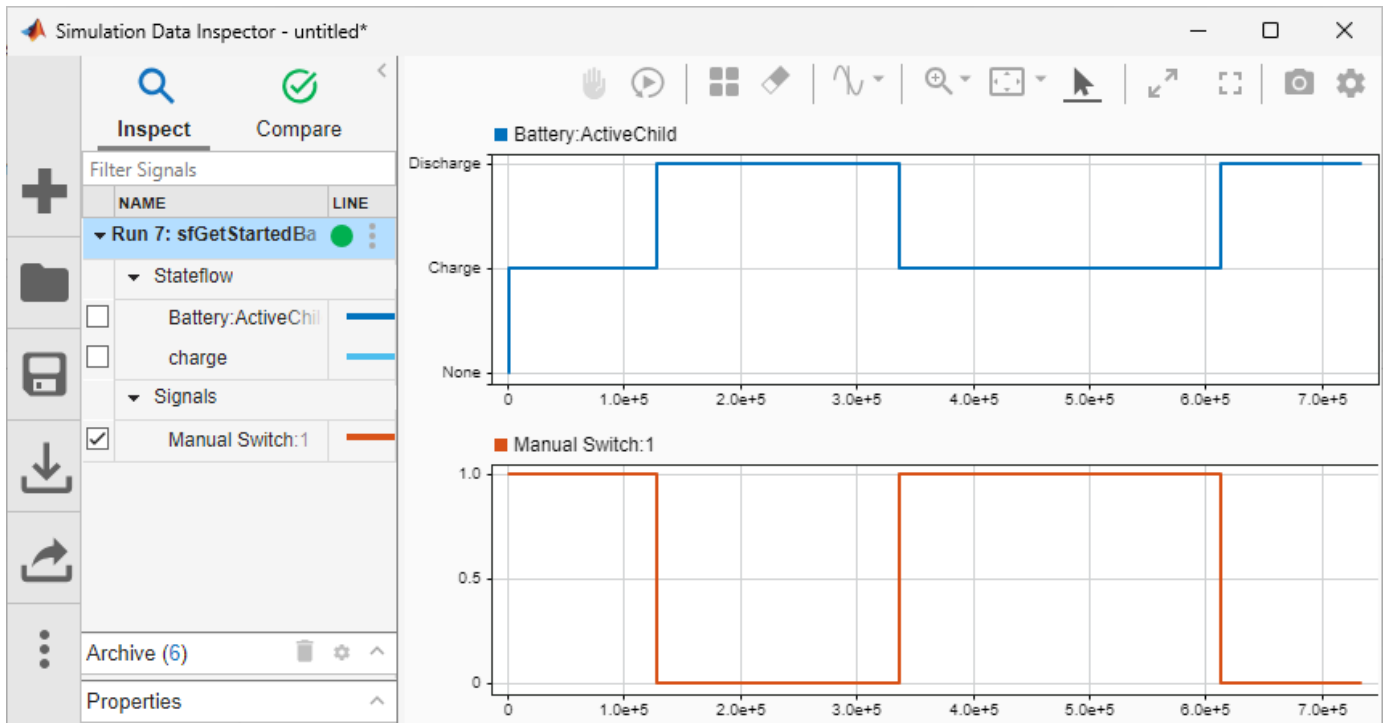
Log States and Data

The battery system has these requirements:

- When `isCharging` is true, charge the battery. When `isCharging` is false, discharge the battery.
- The charge percentage of the battery must remain between 100% and 0%.

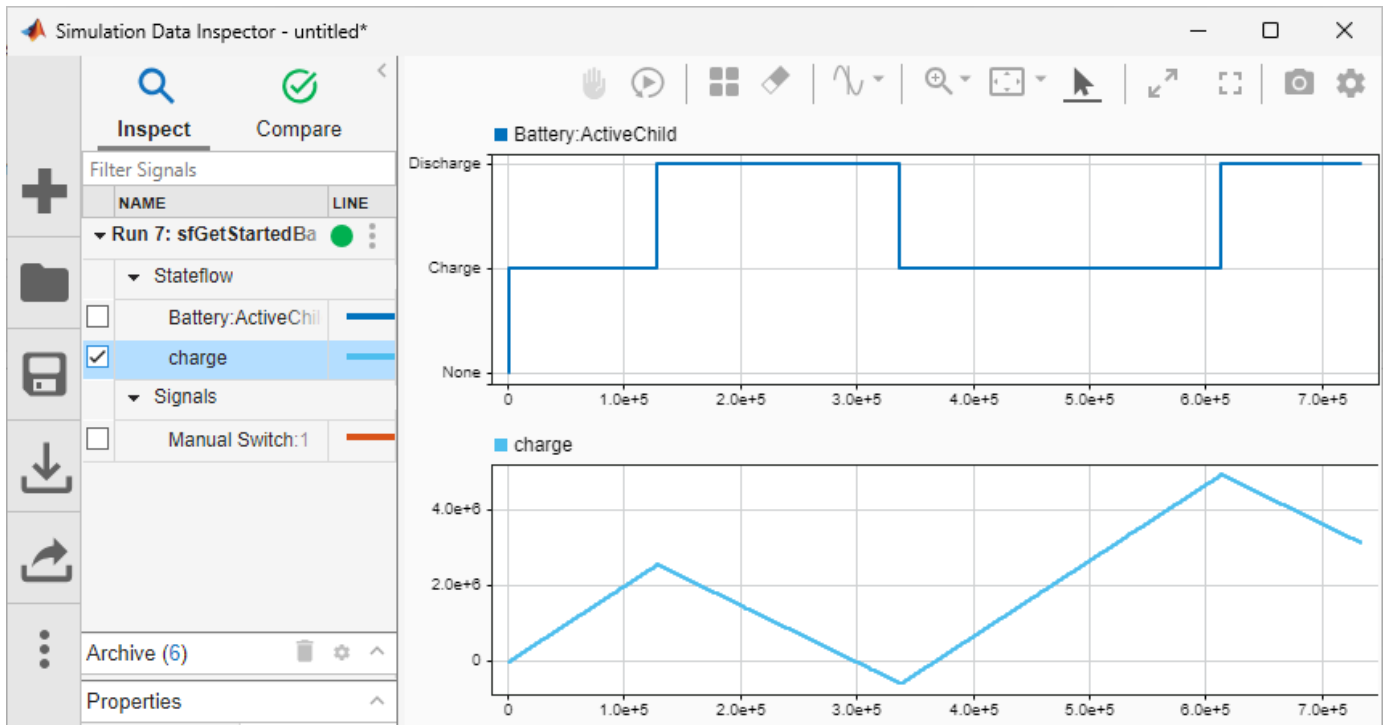
Verify the requirements by logging the active state of the chart and the value of the `charge` input to the Simulation Data Inspector.

- 1 To log the active state of the chart, in the **Simulation** tab, in the **Prepare** section, click **Log Active State**. A logging badge  appears in the lower-left corner of the canvas.
- 2 In the **Symbols** pane, right-click **charge** and click **Inspect**. The Property Inspector opens.
- 3 In the Property Inspector, expand the **Logging** section and select **Log signal data**.
- 4 Return to the Simulink Editor.
- 5 Right-click the signal line from the Manual Switch block to the chart and click **Log Selected Signals**.
- 6 To simulate the model, in the **Simulation** tab, click **Run**. The model simulates until you press **Stop**.
- 7 To transition to the **Discharge** state, double-click the Manual Switch block. Wait several seconds and toggle the Manual Switch block again. Repeat several times. On the final toggle, return the Manual Switch block to 0.
- 8 In the **Simulation** tab, click **Stop** to end the simulation.
- 9 In the **Simulation** tab, in the **Review Results** section, click **Data Inspector**.
- 10 In the Simulation Data Inspector, click Visualization and layouts . In the **Basic layouts** section, click the 2x1 layout .
- 11 To display the active state of the chart, click the top graph. Then, in the **Inspect** tab, expand the **Stateflow** section and select **Battery:ActiveChild**.
- 12 To display the output of the Manual Switch block, click the bottom graph. Then, in the **Inspect** tab, expand the **Signals** section and select **Manual Switch:1**.



When you toggle the Manual Switch block, the chart transitions between Charge and Discharge as expected.

- 13 To display the value of charge, click the bottom graph. Then, in the **Inspect** tab, select charge and clear Manual Switch:1.



The battery contradicts the requirements by charging above 100% and below 0%.

Set Breakpoints

Set breakpoints on states and transitions to identify the time step in which the design problem occurs.

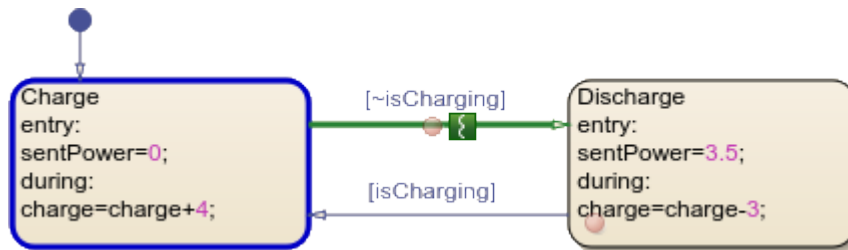
- 1 In the chart, right-click the **Discharge** state and click **Set Breakpoint**. A red circular breakpoint badge appears on the state. By default, breakpoints on a state trigger when the charts enters or remains in the state.
- 2 To disable the breakpoint from triggering when the chart enters the state, click the breakpoint badge. In the **Discharge Breakpoints** dialog box, clear the **On State Entry** trigger.
- 3 Right-click the transition from **Charge** to **Discharge** and click **Set Breakpoint**. The breakpoint triggers when the chart moves along the transition.
- 4 In the **Debug** tab, click **Breakpoints List**.

In the **Breakpoints List** pane, you can enable or disable breakpoints and change the conditions under which they trigger.

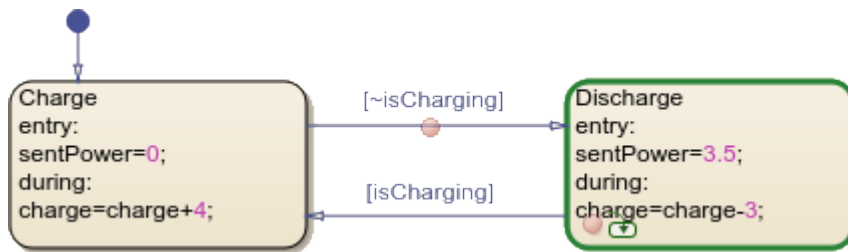
- 5 Specify that the **Discharge** breakpoint triggers only when the battery charge is negative. In the table row for the **Discharge** state, double-click the **Condition** column and enter $\text{charge} < 0$.

Breakpoints List				
+ ● ● ▼ ⌂ 05 ? ☐ Debug Stateflow charts and MATLAB Function blocks ☒ Pause within time step Filter				
Enabled	Source	Source Type	Condition	Hits
☒	sfGetStartedLogging/.../[~isCharging]	Transition Valid		0
☒	sfGetStartedLogging/.../Discharge	State During	charge<0	0

- 6 Simulate the model. The simulation pauses on the transition between Charge and Discharge.



- 7 To resume the simulation, in the **Simulation** tab, click **Continue**. The simulation pauses on the step when charge becomes negative.



- 8 Click **Continue** again. Because charge is still negative, the Discharge breakpoint triggers on the next step.
- 9 Stop the simulation.
- 10 To remove the breakpoints, right-click each breakpoint badge and select **Clear Breakpoint**.

In the next step of the tutorial, you fix the design problem by adding nested operating modes that slow or stop charging or discharging when the battery reaches certain charge levels.

See Also

Related Examples

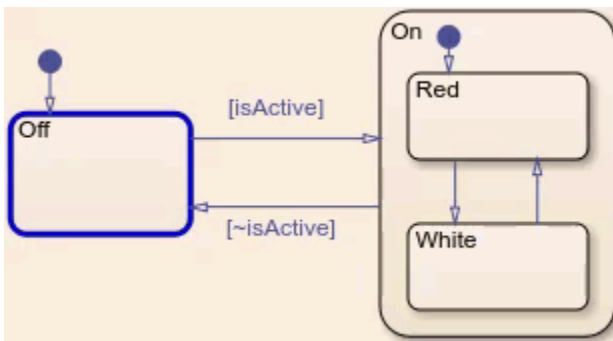
- “Monitor State Activity Through Active State Data”
- “Set Breakpoints to Debug Charts”
- “Inspect and Modify Data and Messages While Debugging”

Create Parent and Child Operating Modes

In Stateflow® charts, you can create nested operating modes by placing a state inside the border of an existing state. The existing state is called a *parent state*, and the states it contains are called *child states*.

The contents of the parent state act like a smaller chart. Similar to a chart, the parent state contains a default transition. When the parent state becomes active, the child state connected to the default transition also becomes active. Then, the chart transitions between the child states. When the parent state deactivates, the child states also deactivate.

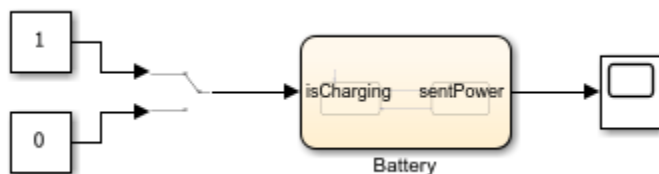
For instance, in this model of the emergency lights on an ambulance, the **On** state has the child states **Red** and **White**. When **On** is active, the ambulance switches between the colored lights, starting with the **Red** state. When **On** deactivates, both lights deactivate.



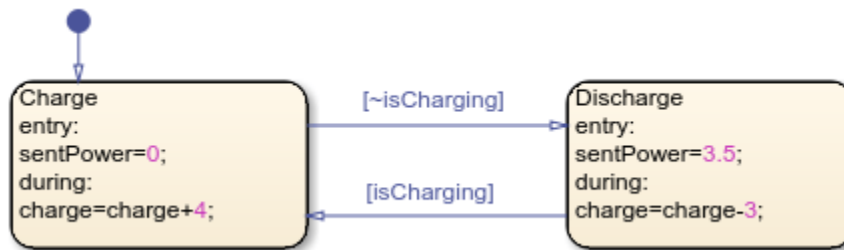
In this example, you use child states to model the variable charging rates in a rechargeable battery system.

Open Model

The `sfGetStartedBattery` model represents the control logic for a rechargeable battery system. To build the model, follow the instructions in the previous steps of the tutorial.



Double-click the Chart block to enter the battery chart.



The states **Charge** and **Discharge** represent the operating modes of the battery system. The input data **isCharging** determines the active state. The output data **sentPower** and local data **charge** represent the output wattage and charge level of the battery.

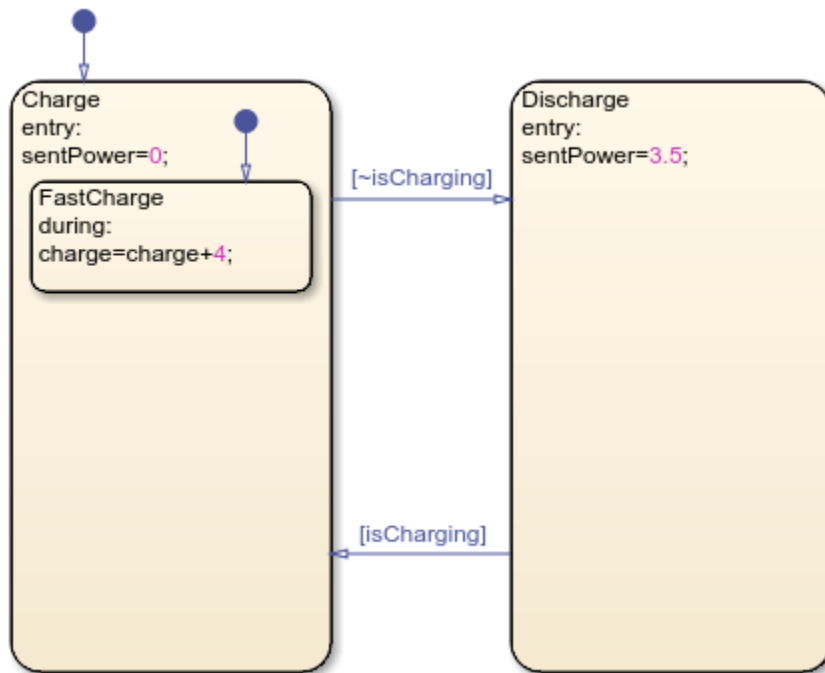
Add Child States to Model Hierarchy

Currently, the battery system continues charging when full and discharging when empty. To represent full, empty, and intermediate charging modes, the battery system requires additional operating modes:

Parent Mode	Child Mode	Actions	Exit Condition
Charge	FastCharge	While active, increase the charge level at a rate of 4%	When the charge level is greater than 80%, move to SlowCharge
Charge	SlowCharge	While active, increase the charge level at a rate of 1%.	When the charge level is equal to 100%, move to Full
Charge	Full	None	None
Discharge	Powered	While active, decrease the charge level at a rate of 3%.	When the charge level is less than or equal to 3%, move to Empty
Discharge	Empty	On entry, set the output wattage to 0	None.

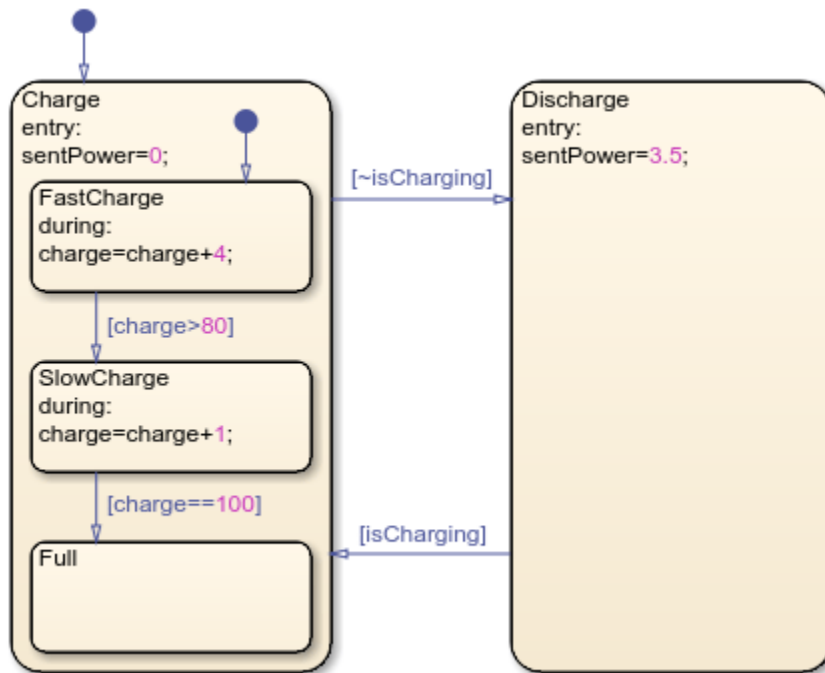
Add a state that represents fast charging.

- 1 In the **Charge** and **Discharge** states, remove the **during** actions.
- 2 Expand the **Charge** and **Discharge** states so that they can contain up to three child states. To resize a state, click any corner and drag outward.
- 3 In **Charge**, add a child state named **FastCharge**. Child states must be fully enclosed by the parent state. The borders of the child and parent states cannot overlap.
- 4 In **FastCharge**, add a **during** action that increments the data **charge** by 4.



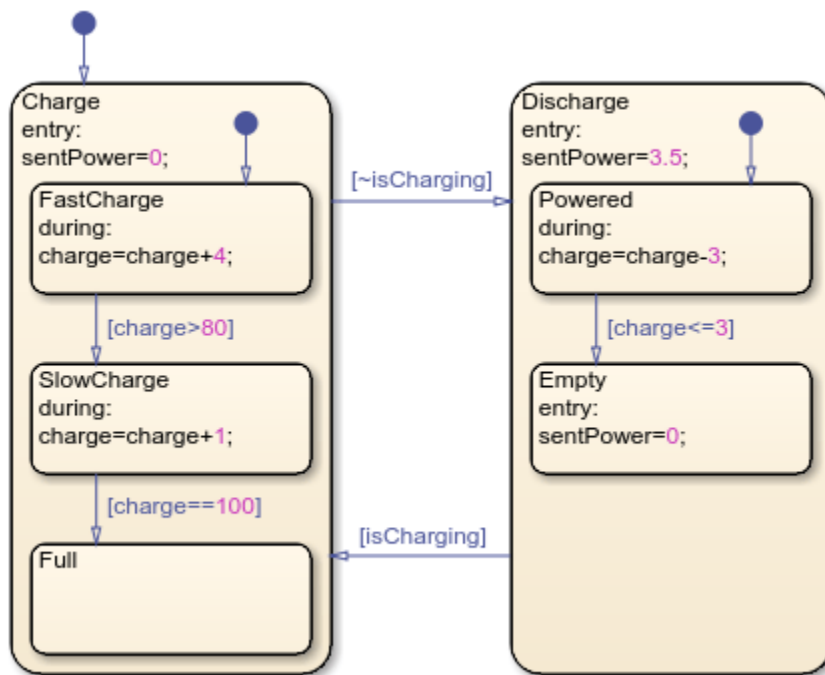
Add two states that represent slow and paused charging.

- 1 Add a child state named **SlowCharge**. Add a `during` action that increments `charge` by 1.
- 2 Add a transition from **FastCharge** to **SlowCharge** that executes when `charge` exceeds 80.
- 3 Add a child state named **Full**.
- 4 Add a transition from **SlowCharge** to **Full** that executes when `charge` equals 100.



Add two states that represent normal and paused discharging.

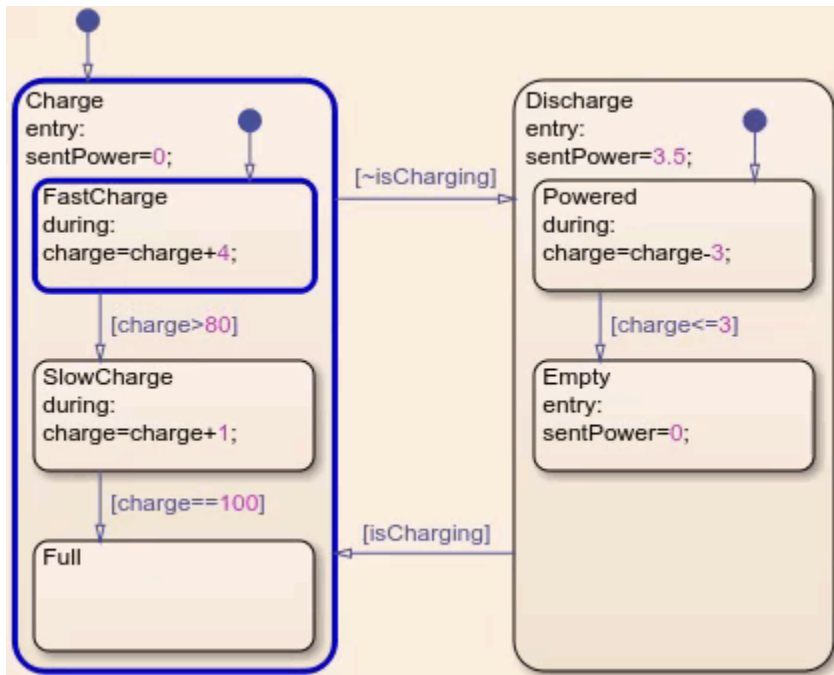
- 1 In **Discharge**, add a child state named **Powered**. Add a during action that decrements `charge` by 3.
- 2 Add another child state named **Empty**. Add an entry action that sets `sentPower` to 0.
- 3 Add a transition between **Powered** and **Empty** that executes when `charge` is less than or equal to 3.



Simulate the Model

Observe how the chart transitions between child states.

- 1 In the Simulink Editor, ensure the Manual Switch block is set to 1.
- 2 In the **Simulation** tab, click **Run** to simulate the model. In the first step, the **Charge** and **FastCharge** both activate. As **charge** increases, the chart transitions to child states **SlowCharge** and **Full**. In the **Symbols** pane, **charge** never exceeds 100.
- 3 Return to the top-level Simulink model. Toggle the value of the Manual Switch block by double-clicking the block.
- 4 Enter the chart. The chart activates the state **Discharge** and the child state **Powered**. When **charge** decreases below 3, the chart transitions from **Powered** to **Empty**. In the **Symbols** pane, **charge** never goes below 0.



The battery model is now more realistic. However, the design still has problems. For example, if the connected device requires less than 3.5 watts of power, the battery output does not change.

In the next step in the tutorial, you create a flow chart that varies the power output of the battery system according to the demand of the connected device.

See Also

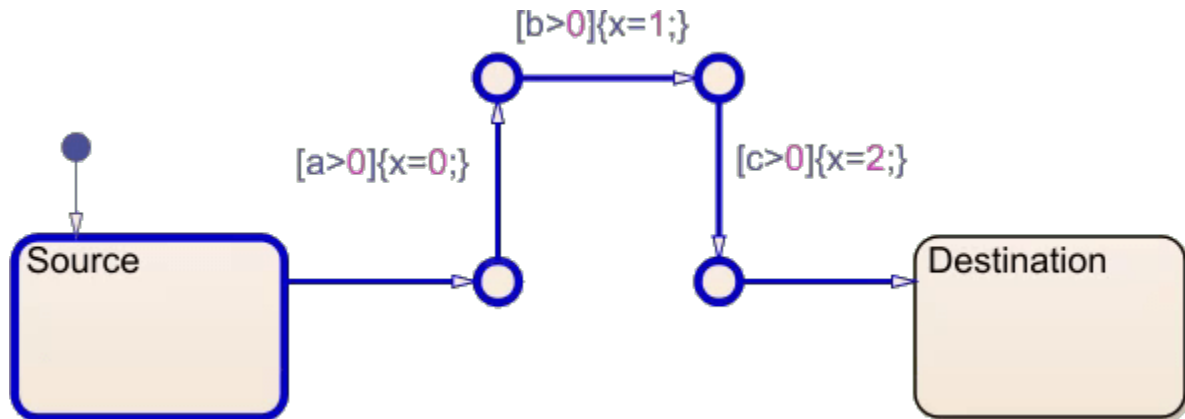
Related Examples

- “Use State Hierarchy to Design Multilevel State Complexity”
- “Move Between Levels of Hierarchy by Using Supertransitions”
- “Execution of a Stateflow Chart”

Connect Transitions by Using Junctions

You can connect multiple transitions into a single path, a branching path, or a flow chart by using junctions.

In Stateflow®, a *path* is one or more transitions that connect a source to a destination. You can create a path with multiple transitions by connecting each transition to a junction. Junctions are graphical objects that represent a decision point, and appear as circles on the canvas. For instance, in the chart below, several transitions and junctions form a single path from one state to another.

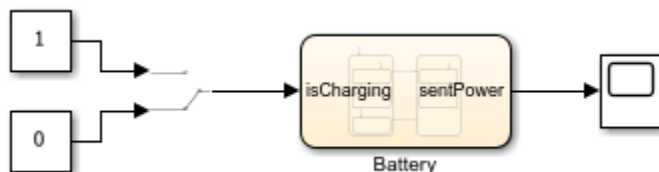


A state or junction can have multiple incoming or outgoing paths. If a state or junction has multiple outgoing paths, the chart numbers the paths in the order they evaluate.

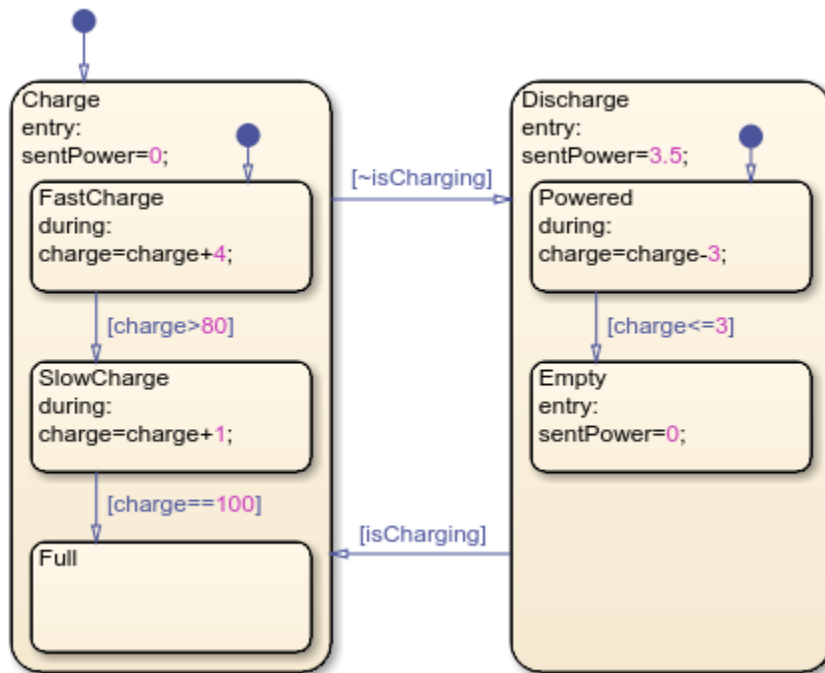
In this example, you use junctions to create a flow chart inside a model of a rechargeable battery system. The flow chart matches the output of the battery system to the demand of the connected device, without exceeding the limits of the battery.

Open Model

To build the model, follow the instructions in the previous step of the tutorial. Alternatively, use the **Open Model** button above and open the `sfGetStartedFlowchart` model.



Double-click the Chart block to enter the Battery chart.



The states **Charge** and **Discharge** represent the operating modes of the battery system. Each state contains child states that represent the charging and discharging rates as the battery fills or empties. The input `isCharging` determines the active state. The data `sentPower` and `charge` represent the output wattage and charge level of the battery.

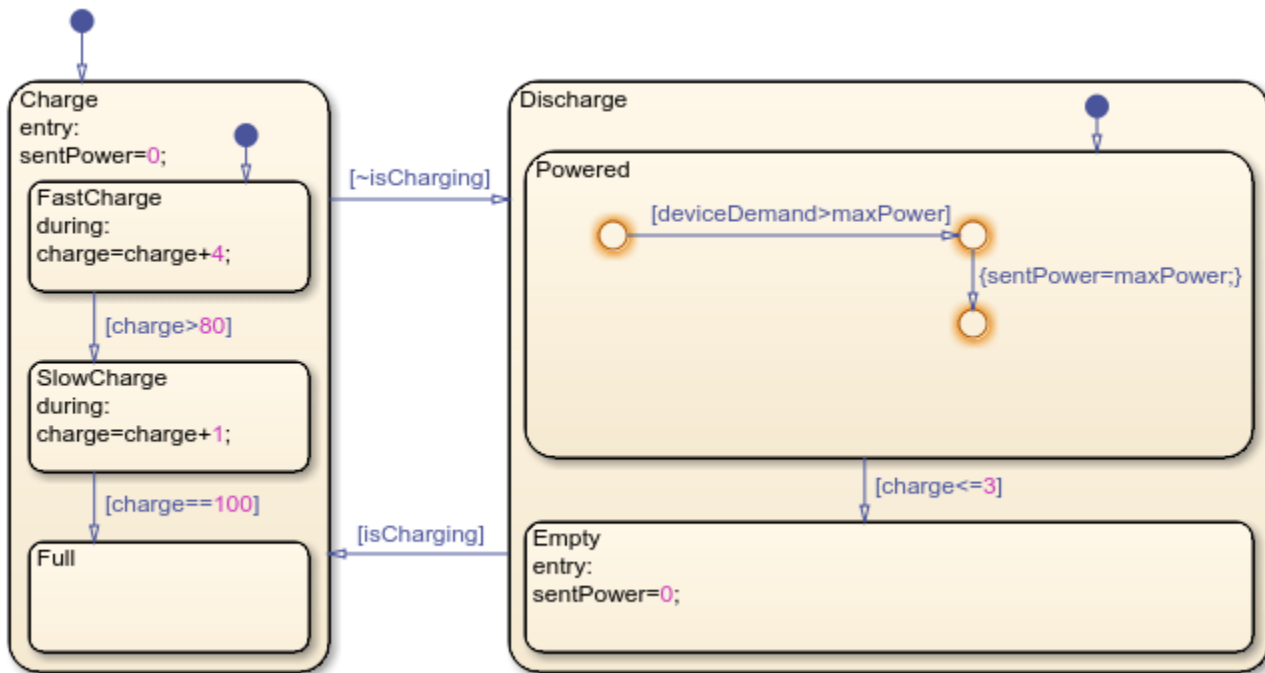
Create Flow Chart

In Stateflow, a flow chart refers to a chart or state whose children consist exclusively of junctions and transitions. All paths in a flow chart must terminate at a single, shared junction. You can create a flow chart at any level of a chart hierarchy.

Currently, when the battery is simultaneously discharging and powered, it always outputs the same wattage. In the **Powered** state, create a flow chart that matches the battery output to the demand of the connected device, without exceeding a maximum value.

Add a path that represents the device demanding more power than the battery can provide.

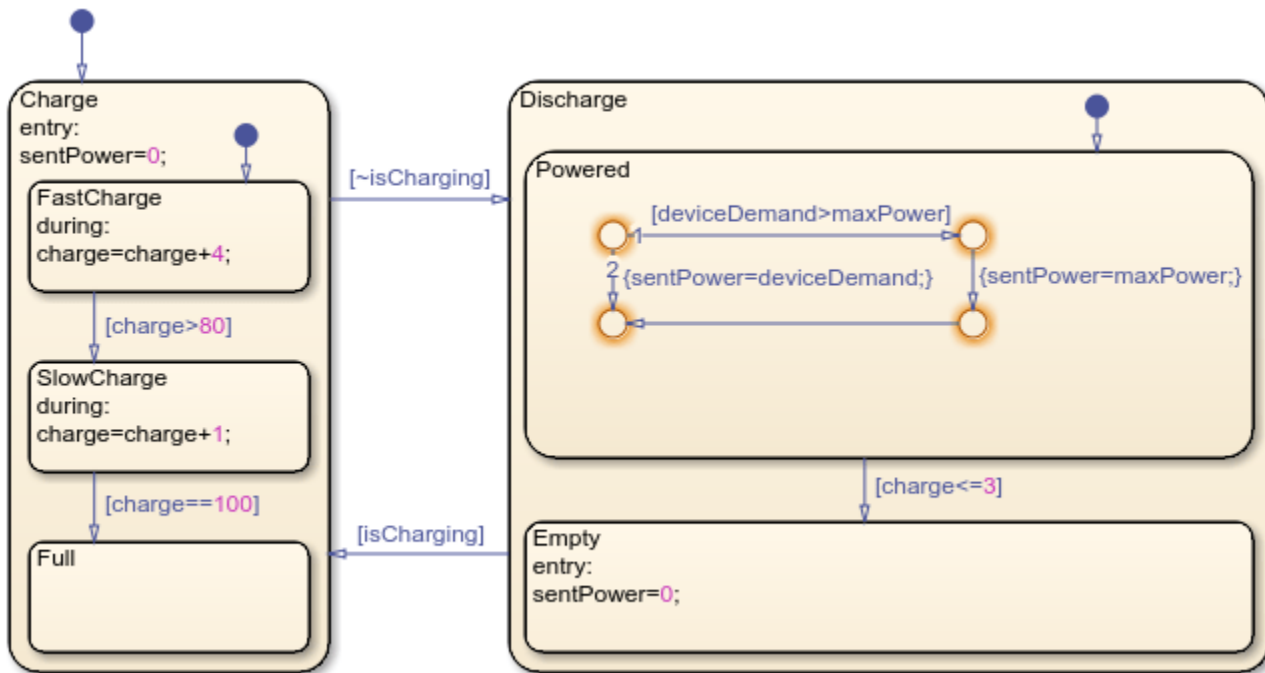
- 1 In the **Discharge** state, remove the entry action.
- 2 In the **Powered** state, remove the during action.
- 3 In the **Powered** state, add the first junction. In the palette, click the **Junction** icon. Place the junction in the **Powered** state.
- 4 Add a second junction to the right of the first junction.
- 5 Add a third junction below the second junction.
- 6 Draw a transition from the first junction to the second. Add the condition `[deviceDemand>maxPower]`.
- 7 Draw a transition from the second junction to the third. Add the action `{sentPower=maxPower;}`.



Add a path that represents power demand within the battery limits.

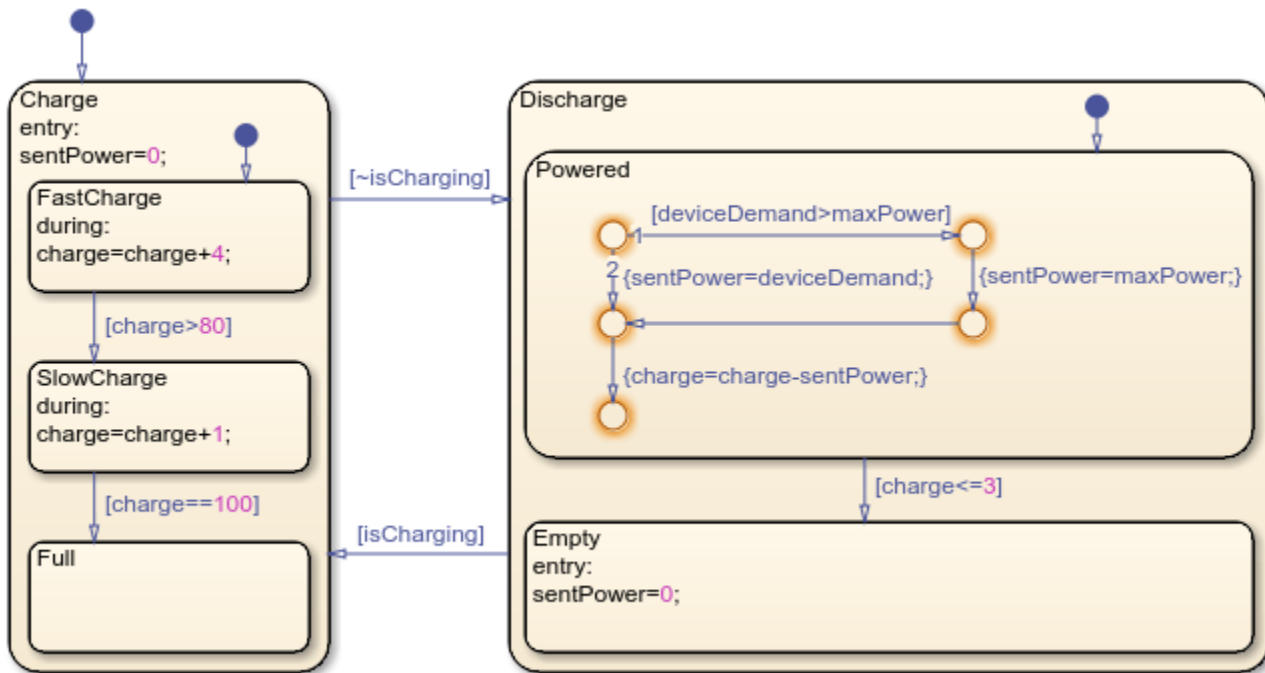
- 1 Add a fourth junction below the first junction.
- 2 Draw a transition from the first junction to the fourth. Add the action `{sentPower=deviceDemand;}`.
- 3 Create a transition from the third junction to the fourth.

The numbers on the first junction indicate the evaluation order. The transition labeled `[deviceDemand>maxPower]` has the order label 1, which indicates it evaluates first. The transition labeled `{sentPower=deviceDemand;}` has the order label 2, which indicates it evaluates second.



A terminal junction is a junction without any outgoing transitions. Connect both paths to a shared terminal junction. In the transition to the terminal junction, decrease the battery charge in proportion to the output wattage.

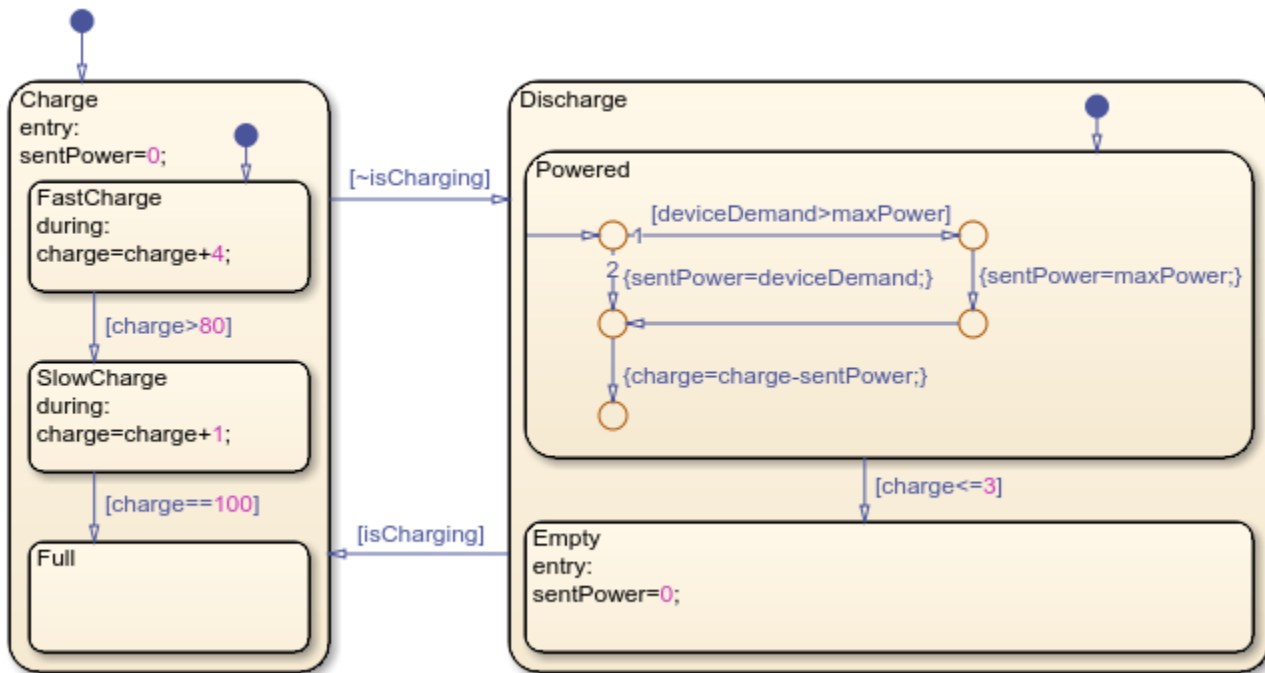
- 1 Add a fifth junction below the fourth junction.
- 2 Create a transition from the fourth junction to the fifth junction. Add the action `{charge=charge-sentPower;}`.



Evaluate During Active Steps by Using Inner Transitions

Inner transitions are the graphical equivalent of a `during` action. When a state contains an inner transition, the inner transition evaluates every step the state is active, but not on steps the state becomes active or inactive. If a state contains both inner transitions and transitions between child states, the inner transitions evaluate first. You can create an inner transition by drawing a transition from the edge of a state to an object within that state.

Draw an inner transition from the edge of the **Powered** state to the first junction.



Define Chart Data

Define the data you created in the flow chart.

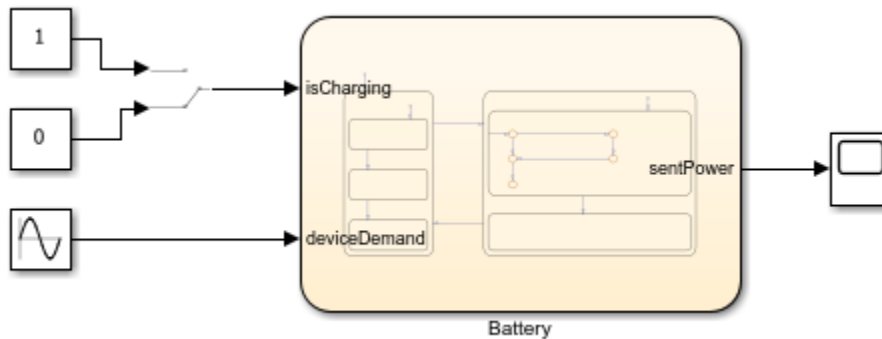
- 1 In the **Symbols** pane, in the `maxPower` row, under **Type**, click the icon and select **Local Data**. Set the **Value** to 3.5.
- 2 In the `deviceDemand` row, under **Type**, click the icon and select **Input Data**.
- 3 In the `charge` row, set the **Value** to 100.

TYPE	NAME	VALUE	PORT
	isCharging		1
	sentPower		1
	charge	100	
	maxPower	3.5	
	deviceDemand		2

Add a block that connects to the `deviceDemand` input port.

- 1 Open the top-level model.
- 2 Add a Sine Wave block. Connect the output port to the **deviceDemand** port of the Battery chart.

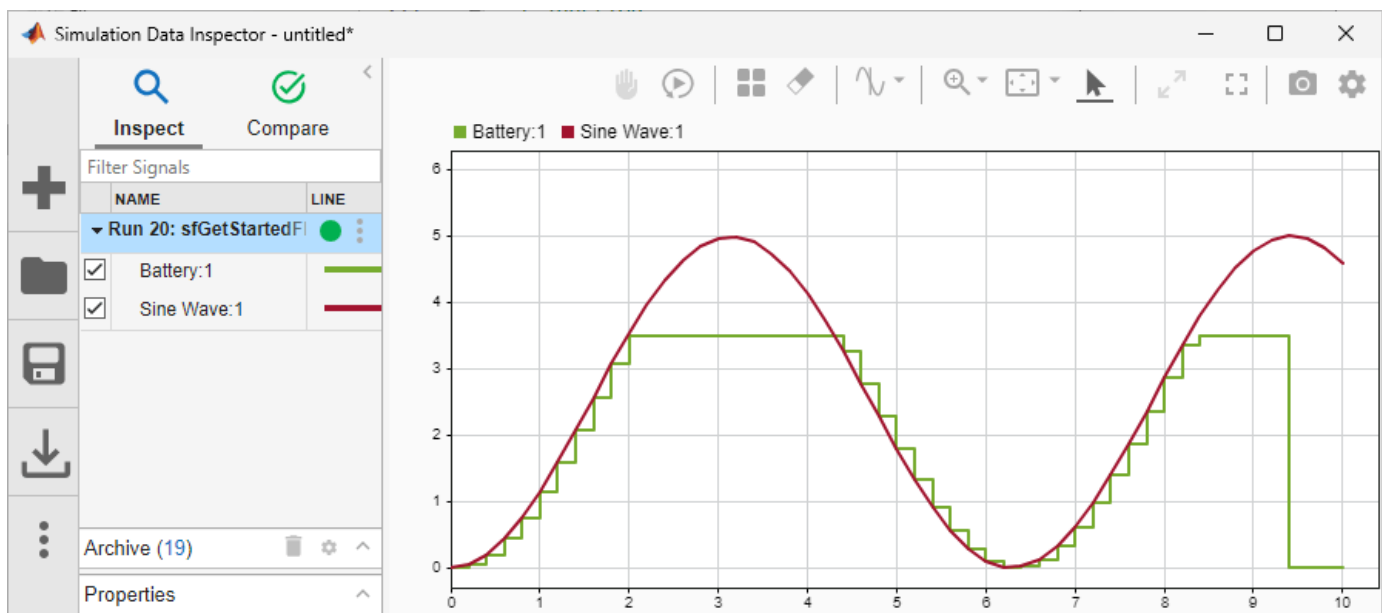
- 3 Specify that the sine wave starts at 0 and reaches a maximum value of 5. Double-click the block to open the block dialog box, then set the **Amplitude** and **Bias** parameters to 2.5. Set the **Phase** parameter to $-\pi/4$.
- 4 Right-click the signal line from the Sine Wave block to the **Battery** chart and click **Log selected signals**. Repeat this step for the signal line from the **Battery** chart to the Scope block.



Simulate Model

Simulate the model and observe the results.

- 1 To simulate the model, in the **Simulation** tab, click **Run**.
- 2 To open the Simulation Data Inspector, in the **Simulation** tab, click **Data Inspector**.
- 3 In the **Inspect** tab, select **Battery:1** and **Sine Wave:1**.



The output wattage of the battery matches the rise and fall of the power draw, to a maximum of 3.5. Near the end of the simulation, the battery runs out of power, and the output wattage drops to 0.

The battery system outputs power as expected. However, if the battery runs out of power, the system can no longer operate.

In the next step of the tutorial, you add a non-rechargeable emergency battery that maintains essential functions if the main battery becomes empty. To model the simultaneous operation of the batteries, you use parallel states and events.

See Also

Related Examples

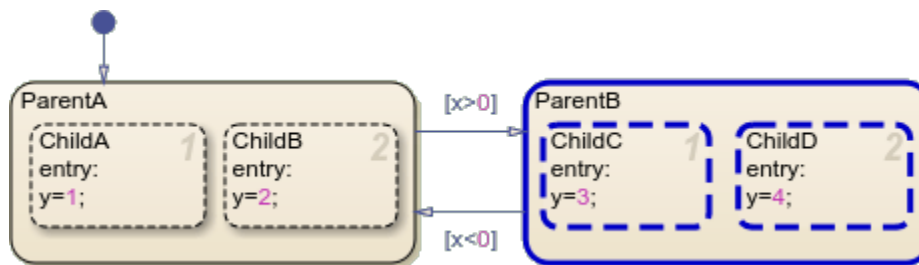
- “Combine Transitions and Junctions to Create Branching Paths”
- “Control Chart Execution by Using Inner Transitions”
- “Execution of a Stateflow Chart”
- “Evaluate Transitions”

Execute States in Parallel

You can set a Stateflow® chart to activate every state in a level of hierarchy on the same step by using parallel decomposition. You can use events to allow one state to trigger behavior in another.

When a chart or state that uses parallel decomposition becomes active, every direct child of the chart or state also becomes active. The children, which are called *parallel states*, appear on the canvas with a dashed border.

For instance, in this chart, the states **ParentA** and **ParentB** use parallel decomposition. When each parent state becomes active, both child states also become active.



On each time step, the chart executes actions and evaluates transitions for each parallel state in a level of hierarchy in sequential order, which is indicated by the number in the upper-right corner of each state. For instance, when **ParentB** becomes active, the chart enters both **ChildC** and **ChildD**. Because **ChildC** has an execution order of 1, it sets *y* to 3. Then, on the same time step, **ChildD** overwrites *y* to 4.

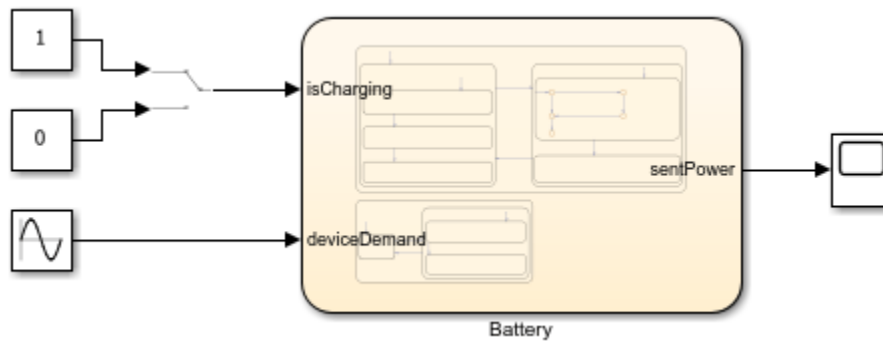
You can change the execution order by right-clicking a state, clicking **Execution Order**, and selecting a new order.

In this example, you use parallel states to finish an incomplete model of a battery system. Then, you use events to create interactions between the states. The battery system contains two independent components:

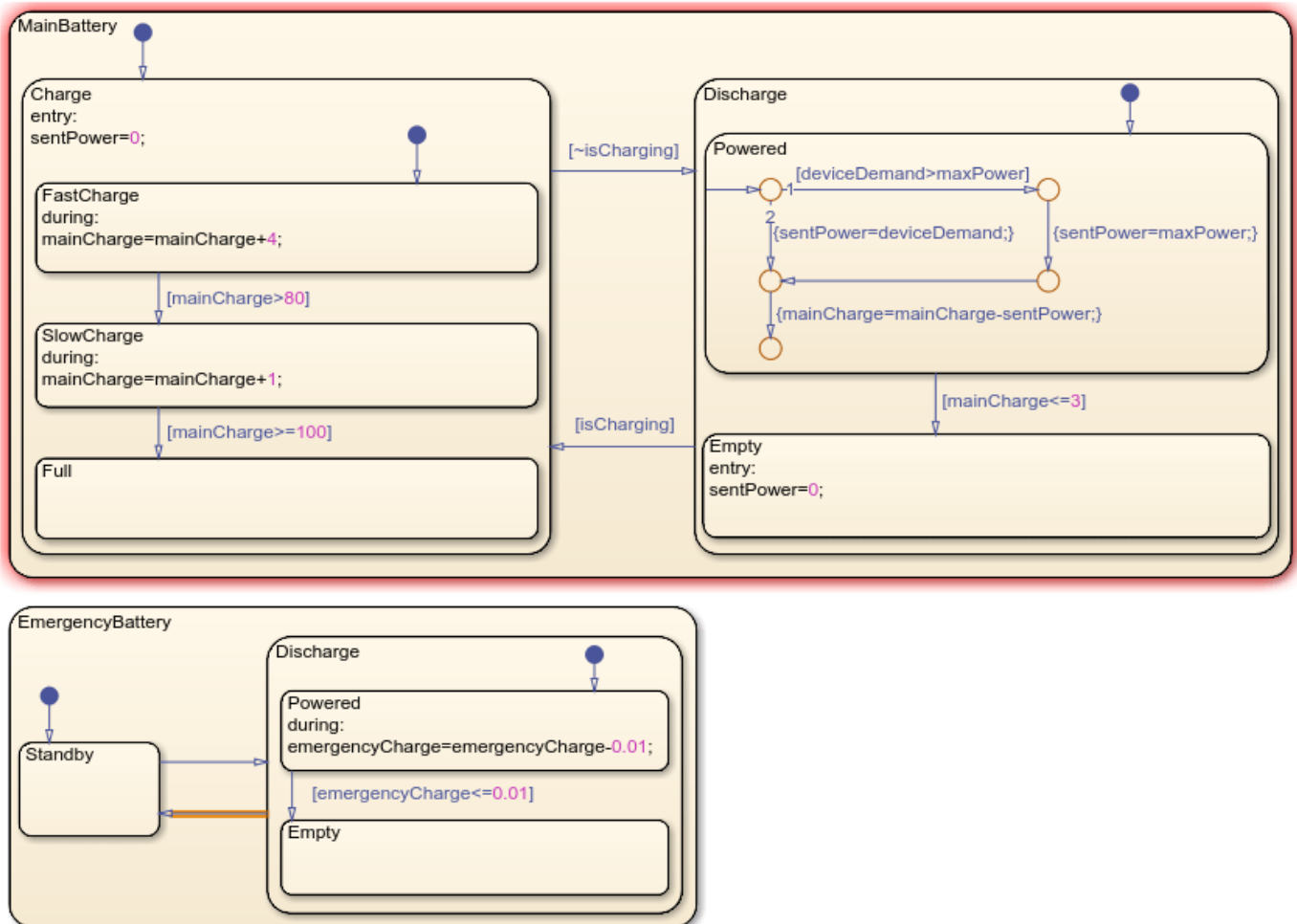
- A rechargeable main battery
- A non-rechargeable emergency battery that maintains essential functions if the main battery becomes empty

Open Model

The `sfGetStartedParallel` model represents the control logic for a rechargeable battery system. To build the model, follow the instructions in the previous steps of the tutorial.



Double-click the Chart block to enter the battery chart.



The chart contains two non-parallel states that represent the main and emergency batteries. Each battery contains nested child states that represent operating modes for charging, discharging, or standing by. Additionally, the batteries use input, output, and local data:

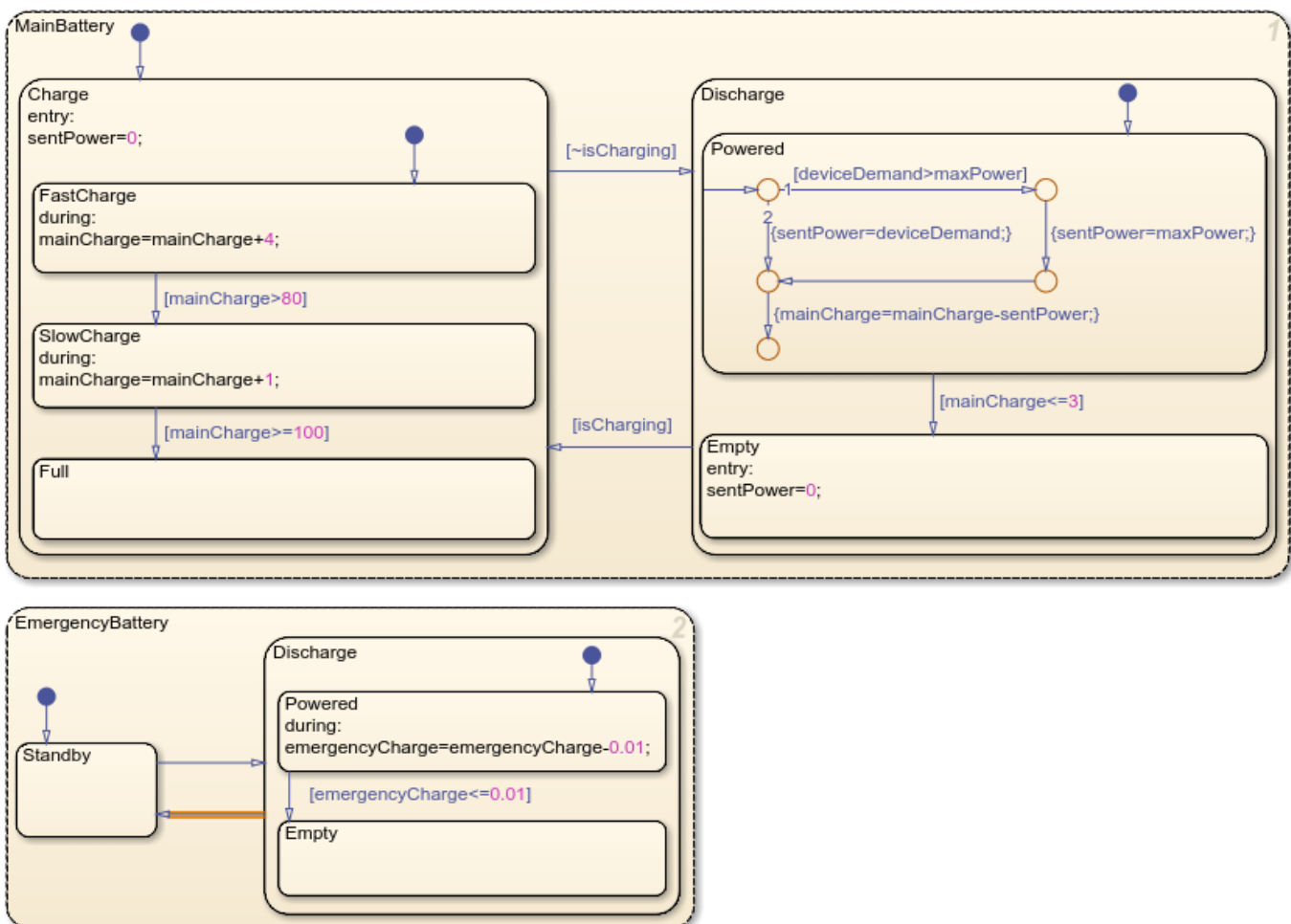
- **isCharging**: An input data that represents whether the battery system is connected to a power source. It determines the active state of the main battery.
- **deviceDemand**: An input data that represents the power demand of the connected device.
- **maxWatts**: A local data that represents the maximum power output of the main battery.
- **sentPower**: A local data that represents the output wattage of the main battery. If **deviceDemand** is less than **maxWatts**, it equals **deviceDemand**. Otherwise, it equals **maxWatts**.
- **mainCharge** and **emergencyCharge**: Local data that represent the charge of each battery as a percent.

For details on the operation of the main battery, see the previous steps in the tutorial.

Enable Parallel Decomposition

You can enable parallel execution for child states by right-clicking the parent component and selecting **Decomposition > Parallel**.

Because the **MainBattery** and **EmergencyBattery** states do not have a parent state, the chart is considered the parent. Enable parallel decomposition for the chart by right-clicking an empty section of canvas and selecting **Decomposition > Parallel**. The borders of the two charts become dashed, which indicates they are parallel states.



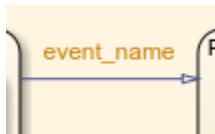
Broadcast and Receive Events

You can enable a chart component to trigger behavior in other components of the chart or model by broadcasting events. For instance, in a chart with two parallel states, you can broadcast an event in one state to cause a transition in the other.

To broadcast an event from a state or transition, use the `send` operator using the format:

```
send(event_name,receiving_state)
```

To receive the event broadcast in a transition, enter the name of the event in the transition label. The event does not require brackets or braces. The transition must be a direct or indirect child of the receiving state.



The project requirements for the battery system state that the emergency battery should discharge when the main battery becomes empty and move to standby when the main battery begins charging. Toggle the operating mode of the emergency battery by broadcasting events from the main battery.

1. In the state `MainBattery`, in the entry action of the child state `Empty`, broadcast an event called `toggleOn` to the state `EmergencyBattery`:

```
entry:
sentPower=0;
send(toggleOn,EmergencyBattery);
```

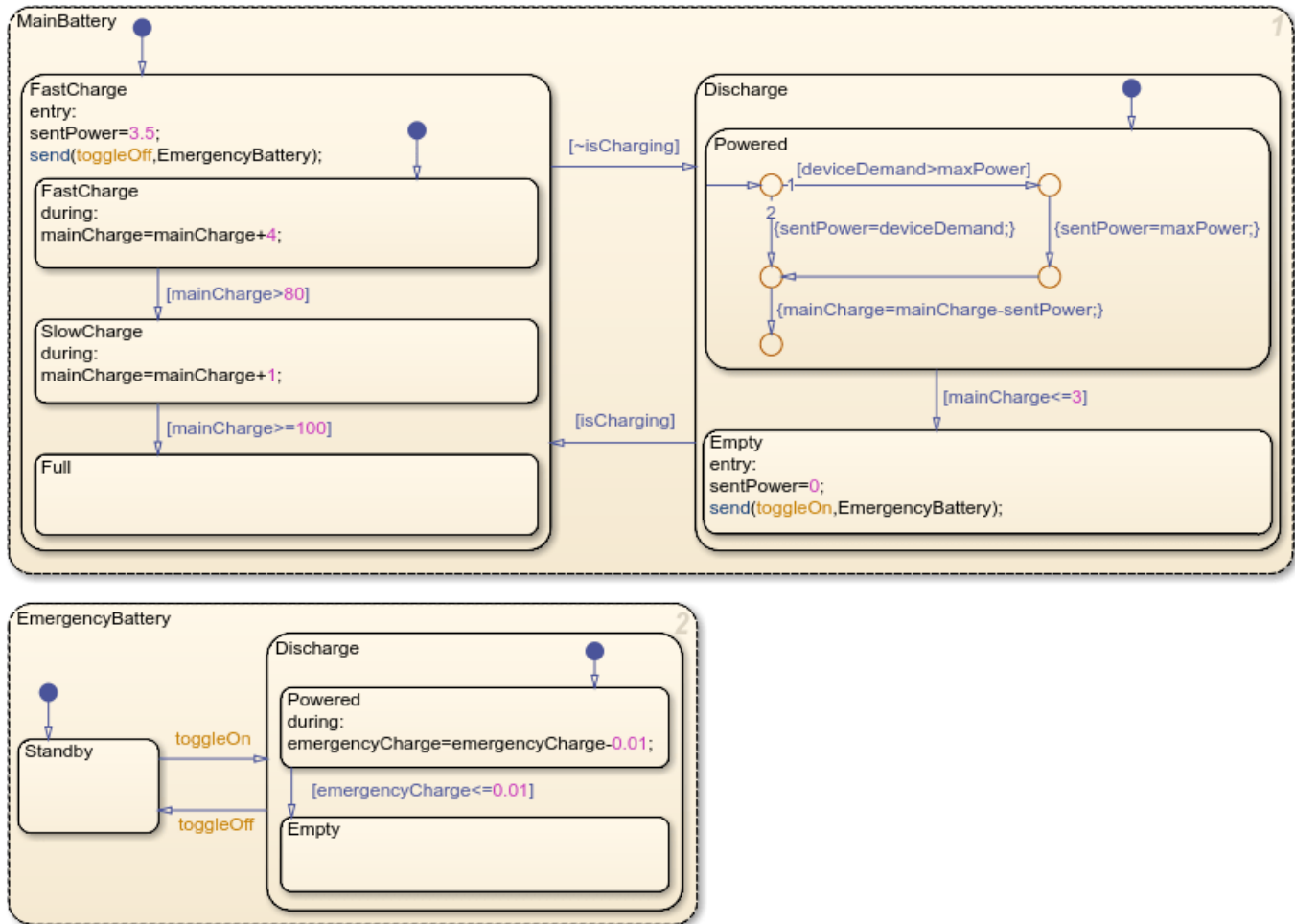
2. In the state `MainBattery`, in the entry action of the child state `Charge`, broadcast an event called `toggleOff` to the state `EmergencyBattery`:

```
entry:
sentPower=3.5;
send(toggleOff,EmergencyBattery);
```

3. In the state `EmergencyBattery`, double-click the transition from `Standby` to `Discharge` and enter the label `toggleOn`.

4. Double-click the transition from `Discharge` to `Standby` and enter the label `toggleOff`.

5. In the **Symbols** pane, click the **Resolve undefined symbols** button to define `toggleOn` and `toggleOff` as local events. In the entry actions and transitions, `toggleOn` and `toggleOff` become orange, which indicates an event.



Simulate Model

The model logs two values to the Simulation Data Inspector:

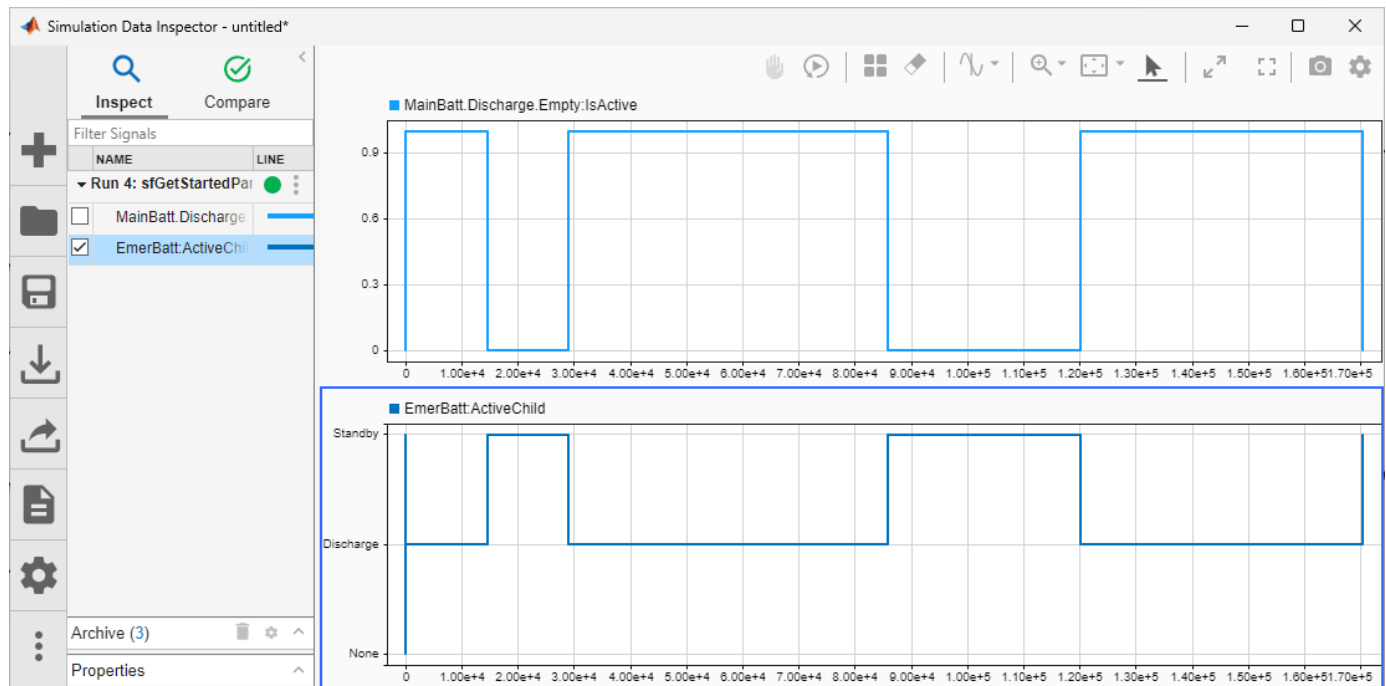
- The activity of the Empty state in MainBattery, where 1 is active and 0 is inactive.
- The name of the active child state of EmergencyBattery.

Simulate the model and observe these values.

- 1 Return to the top-level Simulink model. The Manual Switch block changes the operating mode of the main battery. When the value is 1, the battery charges. When the value is 0, the battery discharges.
- 2 To simulate the model, in the **Simulation** tab, click **Run**. The Chart block shows the execution of the chart.
- 3 Double-click the Manual Switch block to discharge the battery. After a few seconds, when the main battery transitions to Empty, double-click the Manual Switch block to charge the battery. Repeat this process several times.
- 4 To stop the simulation, in the **Simulation** tab, click **Stop**.
- 5 To open the Simulation Data Inspector, in the **Simulation** tab, click **Data Inspector**.

- 6 To show the activity of the Empty state, in the **Inspect** tab, click `MainBattery.Discharge.Empty:IsActive`.
- 7 To show the active child state of the EmergencyBattery state, in the **Inspect** tab, click `EmergencyBattery:ActiveChild`.

When the main battery is empty, the emergency battery discharges. When the main battery leaves the Empty state and begins charging, the emergency battery moves to standby.



See Also

Related Examples

- “Use Events to Execute Charts”
- “Define Exclusive and Parallel Modes by Using State Decomposition”
- “Monitor State Activity Through Active State Data”

Reuse Logic in Charts

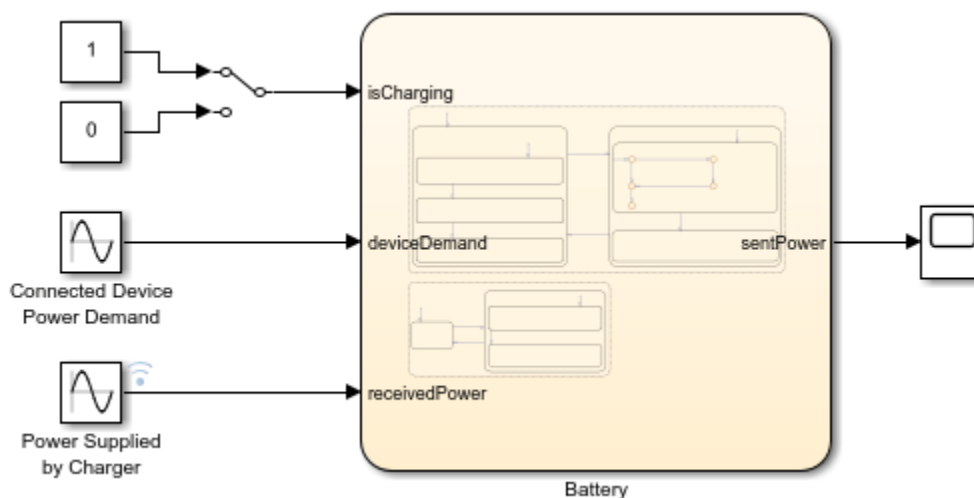
You can reuse logic in Stateflow charts by calling custom functions in states or transitions. Stateflow charts support these custom functions:

- Graphical functions contain a flow chart that consists of junctions and transitions.
- Simulink functions contain a Simulink subsystem.
- MATLAB functions contain MATLAB code.

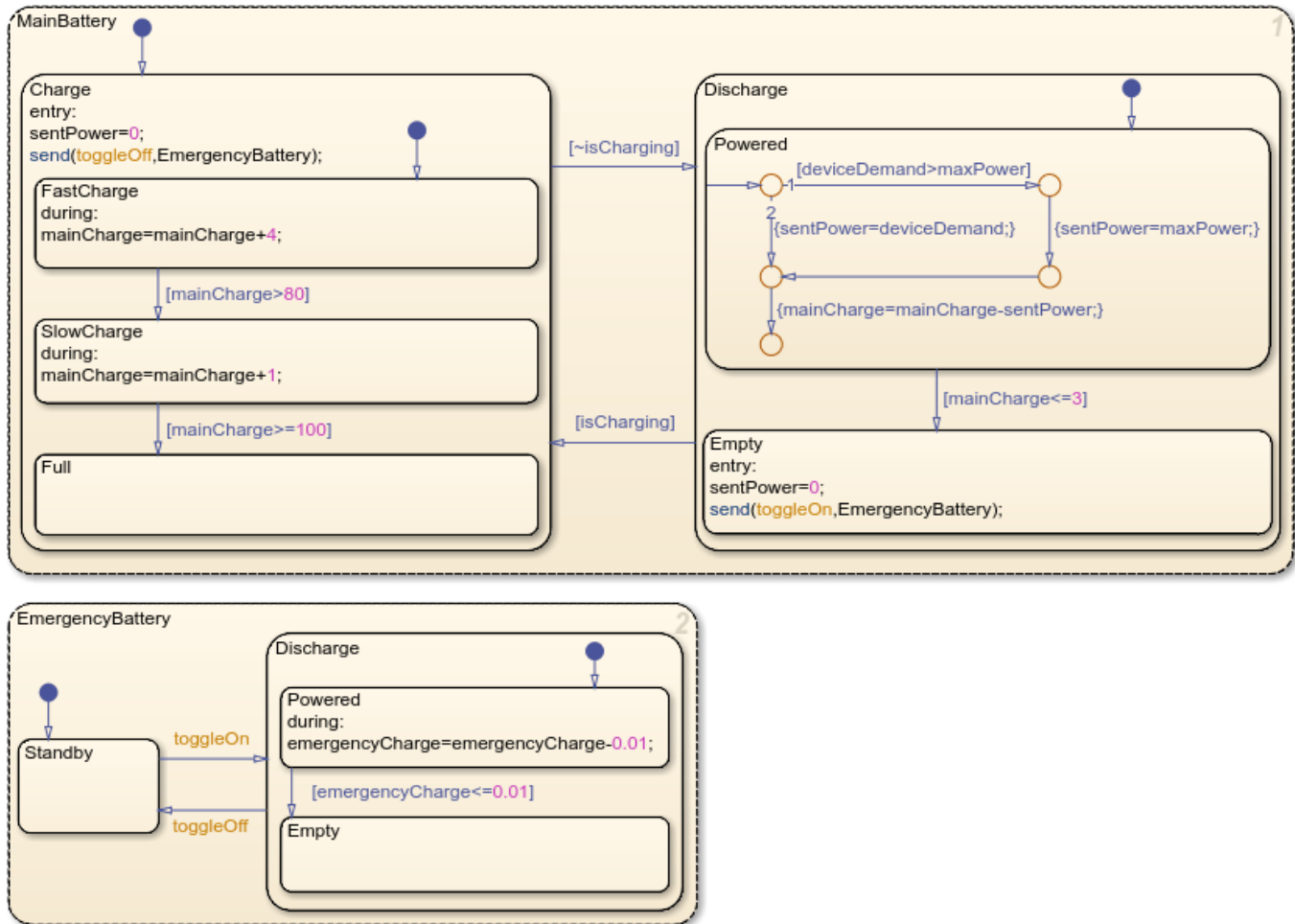
In this example, you create a MATLAB function that enables a model of a rechargeable battery system to handle varying power input when plugged into a charger.

Open Model

The `sfGetStartedFunction` model represents the control logic for a rechargeable battery system. To build the model, follow the instructions in the previous steps of the tutorial.



Double-click the Chart block.



The chart contains two parallel states that represent a rechargeable main battery and non-rechargeable emergency battery. Each battery contains nested child states that represent operating modes for charging, discharging, or standing by.

When the main battery becomes empty, it sends an event that activates the discharging mode of the emergency battery. When the main battery begins charging, it activates the standby mode of the emergency battery.

The chart uses these data:

- `isCharging` — Input data that represents whether or not the battery system is connected to a charger. This data determines the active state of the main battery.
- `deviceDemand` — Input data that represents the power demand of the connected device, in watts.
- `maxPower` — Local data that represents the maximum power output of the main battery, in watts.
- `sentPower` — Output data that represents how much power the main battery delivers to the connected device, in watts.
- `receivedPower` — Input data that represents the power supplied to the main battery by the charger, in watts.

- `mainCharge` and `emergencyCharge` — Local data that represents the charge level of each battery.

Create MATLAB Function

Currently, if the charger provides more or less power, the battery system does not charge faster or slower. Add a MATLAB function that allows the battery to receive variable input power within a minimum and maximum value.

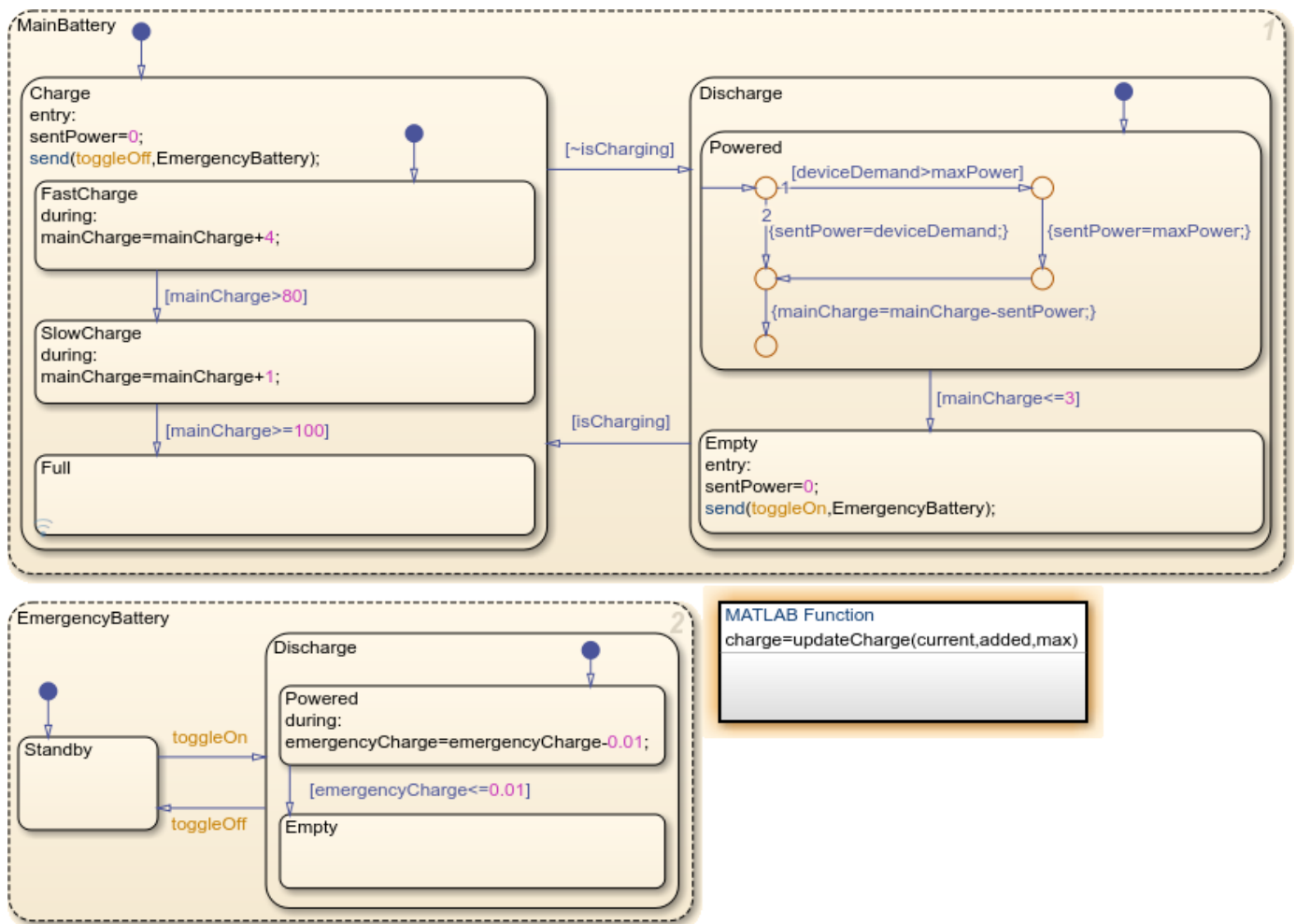
- 1 To create the MATLAB function, in the palette, click the MATLAB function icon . Place the function in an empty section of canvas.
- 2 The editor prompts you to enter a function label. The function label specifies the return values, name, and inputs of the MATLAB function, using this format:

```
[return_val1,return_val2,...] = function_name(input1,input2,...)
```

Function labels with a single return value do not require braces.

Enter a function label that inputs the current charge, the additional charge provided by the charger, and the maximum input charge. Return the updated battery charge.

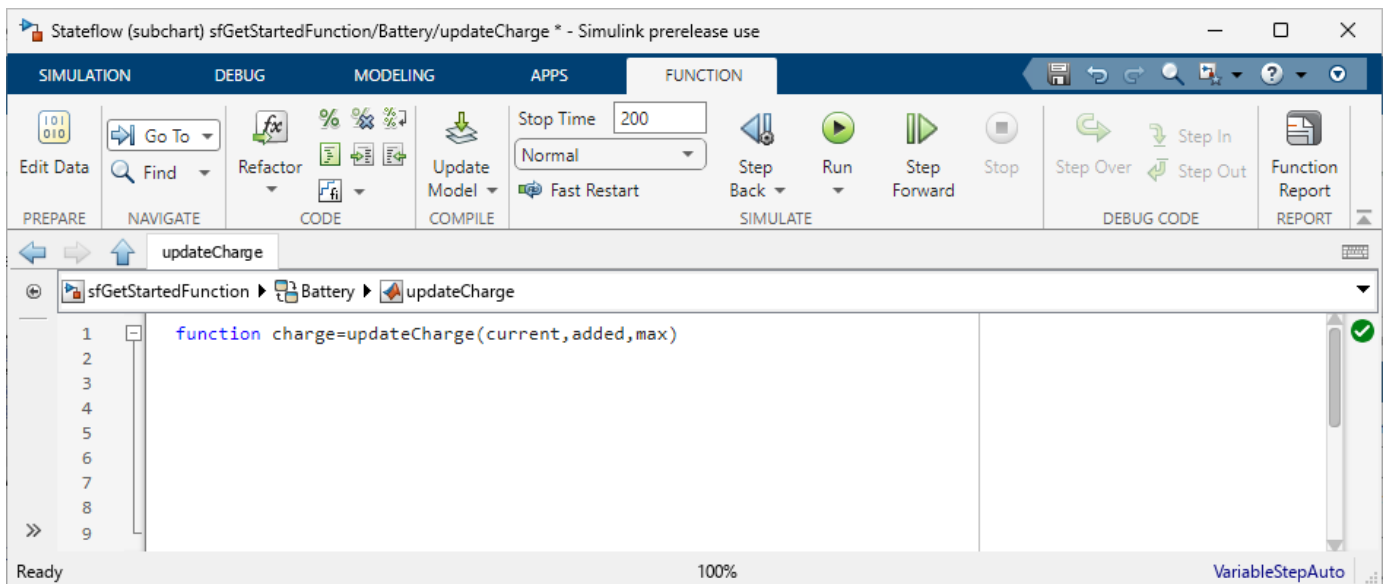
```
charge=updateCharge(current,added,max)
```



- To edit the MATLAB function, double-click any part of the function below the function label. The function editor opens and displays this header:

```
function charge=updateCharge(current,added,max)
```

The function editor copies the header from the function label in the chart. If you change the header, the function label also changes.



- 4 In the function editor, write a function that constrains the battery charging speed between a maximum value and 0.

```

function charge=updateCharge(current,added,max)

if added>max
    charge=current+max;
elseif added<0
    charge=current;
else
    charge=current+added;
end
  
```

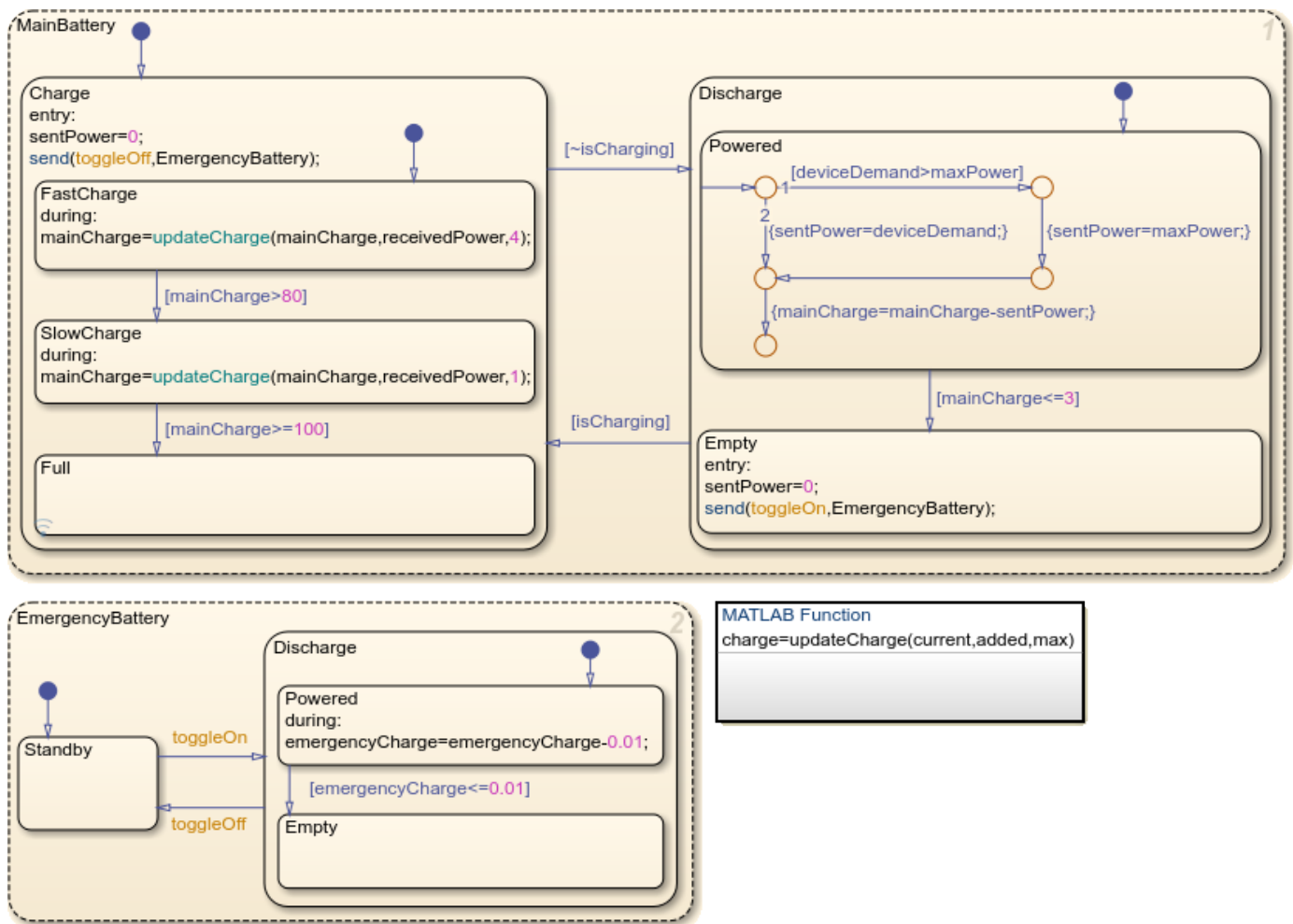
Add MATLAB Function to State Actions

Use the MATLAB function in the states FastCharge and SlowCharge.

- 1 To return to the Stateflow Editor, in the Explorer Bar, click the Up To Parent button .
- 2 In the state FastCharge, modify the during action to use the function. For the maximum input charge, use 4, the value the chart currently adds to mainCharge.


```
during:
mainCharge=updateCharge(mainCharge,receivedPower,4);
```
- 3 In the state SlowCharge, modify the during action to use the function. For the maximum input charge, use 1, the value the chart currently adds to mainCharge.


```
during:
mainCharge=updateCharge(mainCharge,receivedPower,1);
```



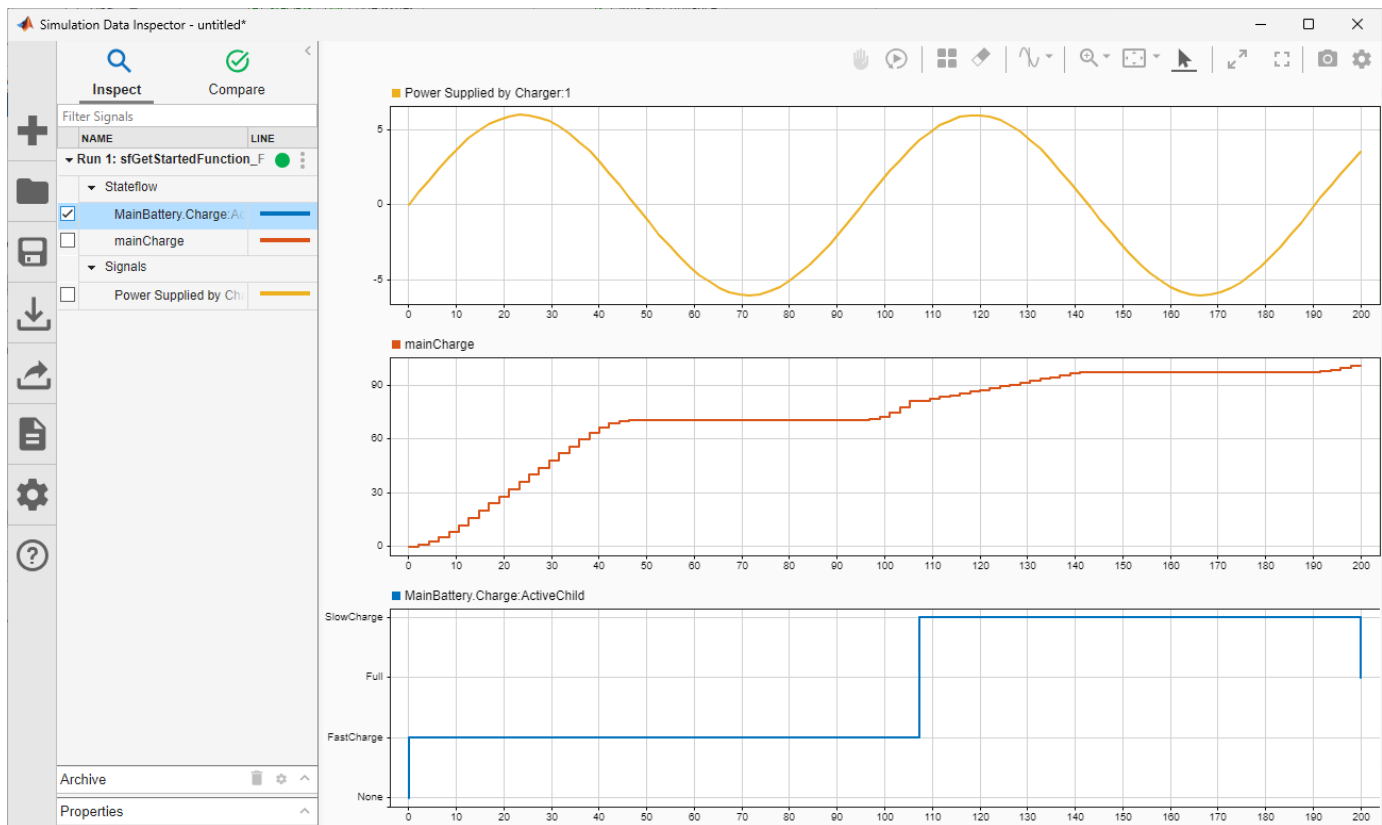
Simulate Model

The model logs these values to the Simulation Data Inspector:

- The power supplied to the battery system by the charger. The value varies between -6 and 6.
- The charge level of the main battery.
- The name of the active charging mode of the main battery.

Simulate the model and use the Simulation Data Inspector to observe how the value of `mainCharge` changes during simulation.

When the main battery charges quickly, the charge level increases and decreases with the power supplied, to a maximum of 4 and a minimum of 0. When the main battery charges slowly, the charge level increases and decreases with the power supplied, to a maximum of 1 and a minimum of 0.



See Also

Related Examples

- "Reuse MATLAB Code by Defining MATLAB Functions"
- "Reuse Logic Patterns by Defining Graphical Functions"
- "Reuse Simulink Functions in Stateflow Charts"
- "Create Reusable Subcomponents by Using Atomic Subcharts"

Create State Transition Tables

In this tutorial, you create a state transition table that models the logic of a rechargeable battery system. The battery system has these requirements:

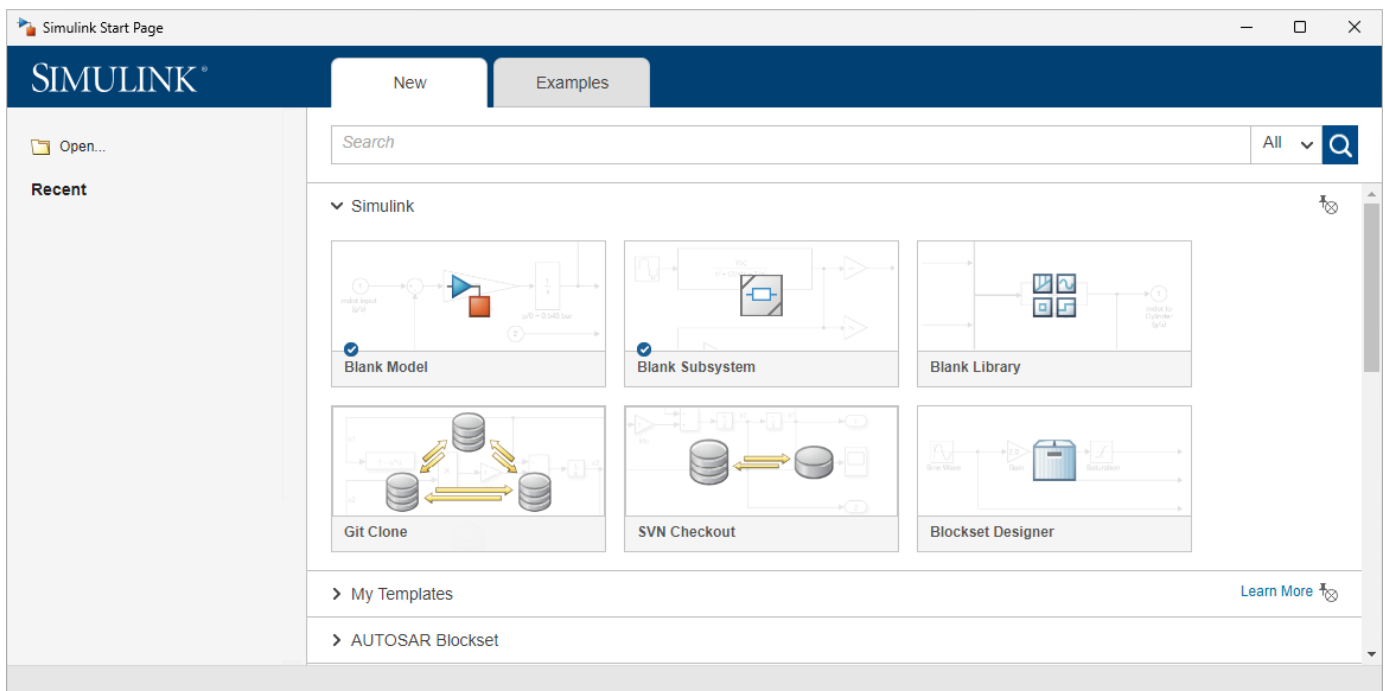
- The battery charges when it is connected to an external power source. Otherwise, the battery discharges until empty.
- The battery capacity charges at a rate of 4% of total charge and discharges rate of 3%.
- When the battery is charging or empty, the battery does not output power. When the battery is discharging, the battery outputs 3.5 watts of power.

To model these requirements, you build a state transition table that contains the states Charge, Discharge, and Empty.

Create Table

Create a new Simulink model that contains an empty State Transition Table block.

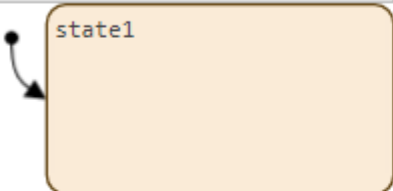
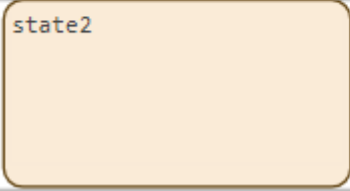
- 1 Start MATLAB. In the MATLAB Toolstrip, in the **Home** tab, click **Simulink**.



- 2 On the start page, in the **Simulink** section, click the **Blank Model** template. Simulink opens and displays an empty model.
- 3 In the model, add a State Transition Table block.



- 4 To view the state transition table, double-click the State Transition Table block.

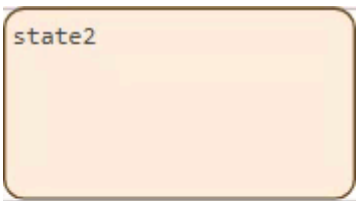
STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 state1		
	\$NEXT ▼	▼
 state2		
	▼	▼

The state transition table contains the states `state1` and `state2`.

Add States

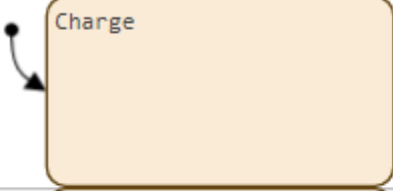
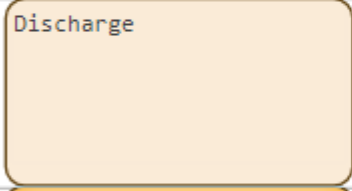
The battery system requires three states: Charge, Discharge, and Empty. To add a third state to the table and rename the existing states:

- 1 Point to `state2`. At the bottom of the state, the **Add state below** link appears. Click the link.



- 2 To edit a state name, click the interior of the state. A text cursor appears. Rename the states to Charge, Discharge, and Empty.

Note State names cannot contain spaces or begin with a number. Each state name must be unique.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 Charge		
	\$NEXT ▼	▼
 Discharge		
	▼	▼
 Empty		
	▼	▼

Select Initial Active State

When the simulation starts, the default transition determines which state becomes active. The default transition icon  indicates the default transition.

To change the default transition, right-click a state and select **Set to default**. Because the requirements state that the battery must start in the charging mode, you do not need to move the default transition.

Move Between States

Transitions determine how and when your chart moves between states. Transitions appear in the **Transitions** column. In this table, each state has two associated transitions, **If** and **Else-If(2)**.

TRANSITIONS	
IF	ELSE-IF(2)
\$NEXT ▼	▼

On each step, the table tests the transitions associated with the active state in left-to-right order. If the table moves along a transition, it does not test the remaining transitions. Each transition has three optional rows that define how it operates:

IF
[Condition]
{Action}
Destination 

- The first row is a condition that must be met before the table can move to the destination state. When you point to a condition, blue square brackets `[]` appear on the left side of the row.

In most cases, the text in the condition row must be surrounded by square brackets.

- The second row is an action that executes when the table moves to the destination state. When you point to a condition, blue curly braces `{ }` appear on the left side of the row.

In most cases, the text in the action row must be surrounded by curly braces.

- The third row is the destination state. When the table meets the condition, the source state becomes inactive and the destination state becomes active. The destination keyword `$NEXT` indicates the chart moves to the state below the source state. The keyword `$PREV` indicates the chart moves to the state above the source state.

If the transition does not have a destination state, the table does not test the transition. If the transition has a destination state but not a condition, the table always moves to the destination state.

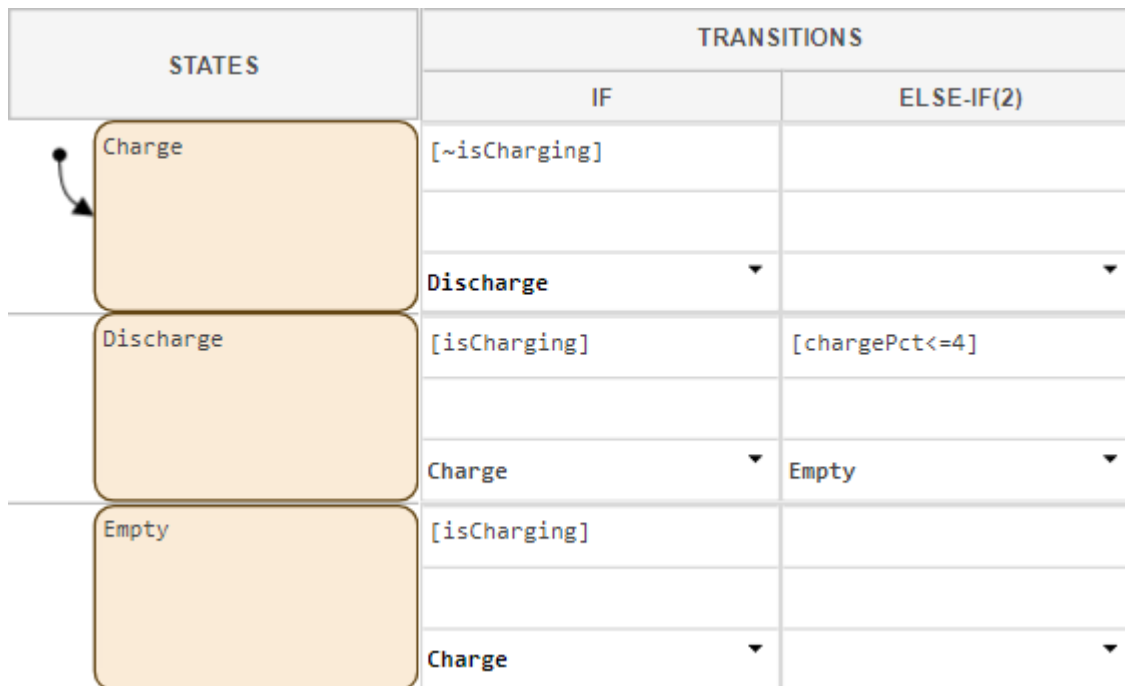
Define Transitions

Use condition rows, action rows, and destination states to define how the battery transitions between operating modes.

- In Charge state, in the **If** transition, click the condition row and enter `[!isCharging]`. This condition represents a battery that is not plugged into a power source.

Click the destination state row and select **Discharge**.
- In the Discharge state, in the **If** transition, enter the condition `[isCharging]` and select the destination Charge. This condition represents a battery that is plugged into a power source.

In the **If-Else(2)** transition, enter the condition `[chargePct<=4]` and select the destination Empty.
- In the Empty state, in the **If** transition, enter the condition `[isCharging]` and select the destination Charge.



Add Executable Code

You can execute code in active states by adding state actions. State actions contain a keyword, followed by a colon and a block of executable code.

In this example, you use three types of state actions.

State Action	Behavior
entry	Executes when the state becomes active.
during	Executes every step a state is active. Does not execute on a step when the state becomes active or becomes inactive.
exit	Executes when the state becomes inactive.

Add state actions that change the battery output and charge according to the operating mode.

- 1 In the Charge state, edit the state label by clicking the state name. Add a new line, then enter the text below. You can add new lines by pressing **Enter**.

```
entry:
sentPower=0;
during:
chargePct=chargePct+3;
```

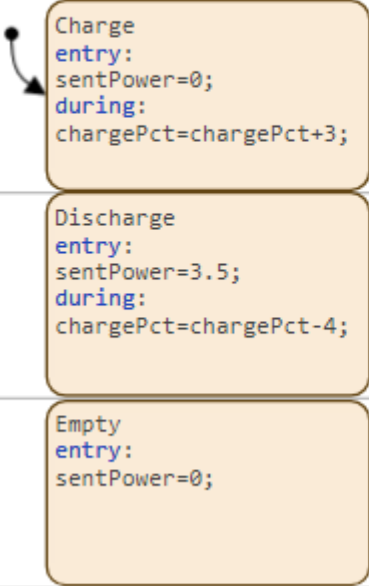
The entry action sets a variable named `sentPower` to 0. This variable represents the battery output in watts. The during action increments a variable named `chargePct` by 3. This variable represents the battery charge as a percent

- 2 In the Discharge state, add an entry action that sets `sentPower` to 3.5 and a during action that decrements `chargePct` by 4.

```
entry:
sentPower=3.5;
during:
chargePct=chargePct-4;
```

- 3 In the Empty state, add an entry action that sets sentPower to 0.

```
entry:
sentPower=0;
```

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 Charge entry: sentPower=0; during: chargePct=chargePct+3;	[~isCharging]	
	Discharge	
Discharge entry: sentPower=3.5; during: chargePct=chargePct-4;	[isCharging]	[chargePct<=4]
	Charge	Empty
Empty entry: sentPower=0;	[isCharging]	
	Charge	

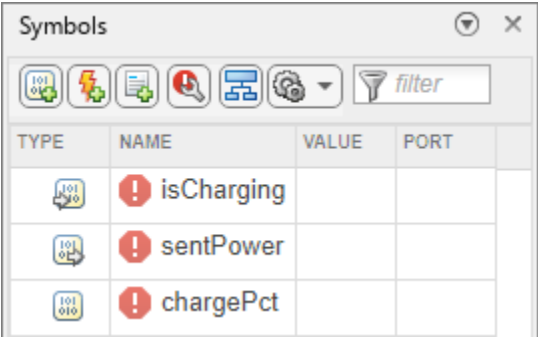
Define Table Data and Share with Simulink Model

When you use a variable in a transition or state, you must define the variable as input data, output data, or local data. In the **Symbols** pane, the warning badge  indicates undefined data.

Icon	Type	Behavior
	Input Data	An input port on the State Transition Table block determines the value of this data. Defining an input data adds an input port to the State Transition Table block. You cannot manually assign values to input data.

Icon	Type	Behavior
	Output Data	The table outputs the value of this data to an output port on the State Transition Table block. Defining an output data adds an output port to the State Transition Table block.
	Local Data	This data stores information that is only accessible in the table.

The table infers the type of each data based on context. For example, in this example, the table infers that `isCharging` is an input data, `sentPower` is an output data, and `chargePct` is a local data.

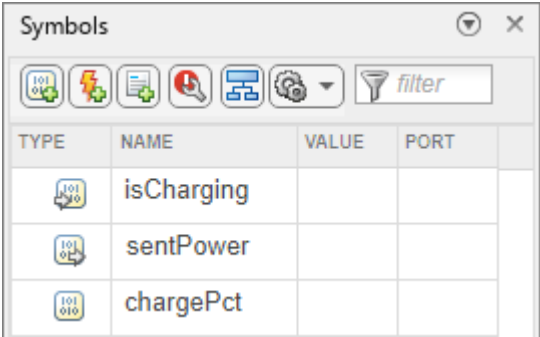


TYPE	NAME	VALUE	PORT
	 <code>isCharging</code>		
	 <code>sentPower</code>		
	 <code>chargePct</code>		

Define the type and value of the table data.

- 1 To accept the inferred data types, in the **Symbols** pane, click the Resolve undefined symbols button . The warning badges next to the undefined data disappear.
- 2 Set the initial charge of the battery. In the **Symbols** pane, the **Value** column specifies the initial value of each data. Data with an undefined value defaults to 0.

In the `chargePct` row, click the **Value** column and enter 50.



TYPE	NAME	VALUE	PORT
	<code>isCharging</code>		
	<code>sentPower</code>		
	<code>chargePct</code>	50	

During simulation, the **Value** column updates as the value of each data changes. When the simulation ends, the **Value** column returns to the original assigned value. You cannot edit data during simulation.

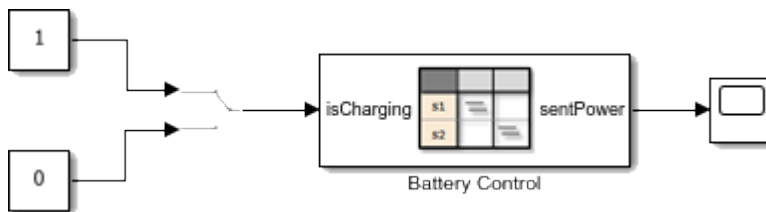
- 3 To return to the top level of the Simulink model, in the explorer bar, click the Up to Parent button . The State Transition Table block has an input and output port. To see the port names, expand the State Transition Table block by clicking a corner of the block and dragging outward.



Connect Simulink Blocks to Table

To complete the model, connect source and sink blocks to the input and output ports of the State Transition Table block.

- 1 To represent a battery system that can connect to or disconnect from an external power source, add a Manual Switch block to the Simulink canvas. Connect the output to the **isCharging** port of the State Transition Table block.
- 2 Add a Constant block with a value of 1. Connect the block to the top input port of the Manual Switch block.
- 3 Add a Constant block with a value of 0. Connect the block to the bottom input port of the Manual Switch block.
- 4 Add a Scope block. Connect the **sentPower** port of the State Transition Table block to the Scope block.
- 5 Name the State Transition Table block **Battery Control**.



Simulate Model

Simulate the completed model.

- 1 In the **Simulation** tab, set **Stop Time** to Inf.
- 2 Double-click to enter the State Transition Table block.
- 3 To simulate the model, in the **Modeling** tab, click **Run**. Observe the blue highlighting around the Charge state. This indicates the state is active.
- 4 Return to the Simulink Editor.
- 5 Toggle the Manual Switch block by double-clicking the block.
- 6 Open the Stateflow Editor. The Discharge state becomes active. Then, when the charge is near zero, the Empty state becomes active.
- 7 To end the simulation, in the **Modeling** tab, click **Stop**.
- 8 Toggle the Manual Switch block to 1.

In the next step of the tutorial, you use active state output, logging, and breakpoints to verify and debug the battery model.

See Also

Related Examples

- “Set Data Properties”
- “Eliminate Redundant Code by Combining State Actions”
- “Model Bang-Bang Controller by Using a State Transition Table”

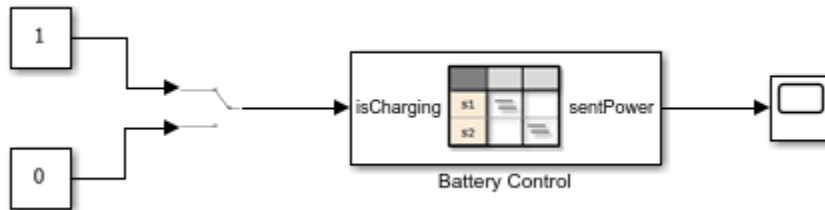
Log, Verify, and Debug State Transition Tables

In state transition tables, you can understand when and how states change during simulation by logging data and states, or by using breakpoints to pause simulation when the table interacts with a state or transition. By combining these techniques, you can verify and debug your state transition table logic.

In this example, you identify a design problem in a rechargeable battery model by logging the state activity and data values of a table. Then, you use breakpoints to identify the state and time step in which the design problem occurs.

Open Model

The `sfGetStartedBatteryTable` model represents the control logic for a rechargeable battery system. To build the model, follow the instructions in the previous steps of the tutorial.



The State Transition Table block `Battery Control` contains the control logic for the system. To determine whether the battery charges or discharges, the table receives a Manual Switch block as an input.

To view the table, double-click the State Transition Table block.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 <pre> Charge entry: sentPower=0; during: chargePct=chargePct+3; </pre>	[~isCharging]	
	Discharge	
<pre> Discharge entry: sentPower=3.5; during: chargePct=chargePct-4; </pre>	[isCharging]	[chargePct<=4]
	Charge	Empty
<pre> Empty entry: sentPower=0; </pre>	[isCharging]	
	Charge	

The states Charge, Discharge, and Empty represent the operating modes of the battery system. The table uses these data:

- **isCharging**: An input that determines whether the battery should charge or discharge.
- **chargePct**: A local data that represents the charge level of the battery as a percent of total charge.
- **sentPower**: An output that determines how many watts of power the battery sends at any given time.

Log States and Data

The battery system has these requirements:

- When **isCharging** is true, charge the battery. When **isCharging** is false and the battery has power, discharge the battery until empty.
- The charge percentage of the battery must remain between 100% and 0%.

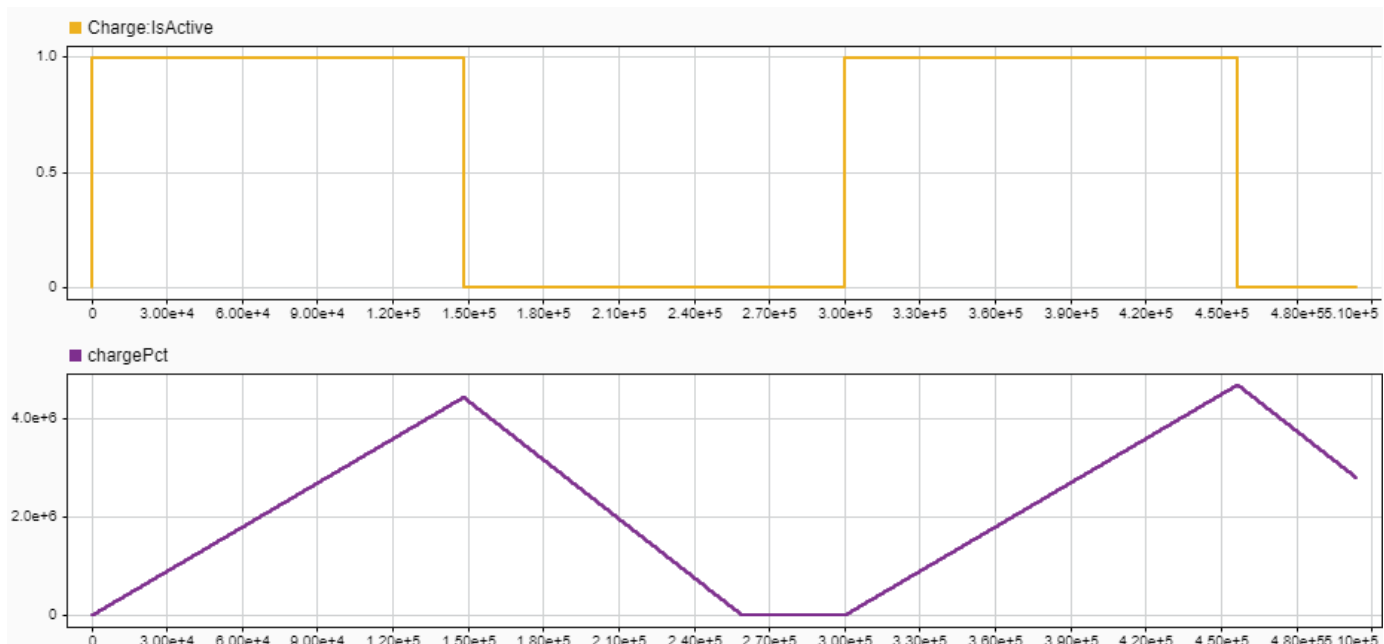
Verify the requirements by logging the active state of the chart and the value of the **chargePct** input.

- 1 Click the Charge state. In the **Simulation** tab, click **Log Self Activity**.
- 2 In the **Symbols** pane, right-click **chargePct** and click **Inspect**. The Property Inspector opens.
- 3 In the Property Inspector, expand the **Logging** section and select **Log signal data**.
- 4 To simulate the model, in the **Simulation** tab, click **Run**.
- 5 To simulate unplugging the battery, double-click the Manual Switch block. Wait several seconds and toggle the Manual Switch block again.
- 6 In the **Simulation** tab, click **Stop** to end the simulation.

Now, display the logged data in the Simulation Data Inspector.

- 1 In the **Simulation** tab, in the **Review Results** section, click **Data Inspector**.
- 2 In the Simulation Data Inspector, click the Visualization and layouts  icon. In the **Basic layouts** section, click the 2x1 layout .
- 3 To display the status of the Charge state, click the top graph. Then, in the **Inspect** tab, select Charge:IsActive.
- 4 To display the value of the chargePct data, click the bottom graph. Then, in the **Inspect** tab, select chargePct.

The top graph shows when the battery charges and discharges. The bottom graph shows the battery charge, which increases beyond 100%.



Set Breakpoints

Set breakpoints on states and transitions to identify the time step in which design problems occur.

- 1 Right-click the Charge state and click **Set Breakpoints > During State**. A breakpoint badge  appears in the top-right corner of the state.
- 2 In the row for the Charge state, right-click any row in the **If** column and select **Set Breakpoints > When Transition Is Valid**. A breakpoint badge  appears on the right side of the condition row.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
Charge entry: sentPower=0; during: chargePct=chargePct+3;	[~isCharging]	
Discharge entry: sentPower=3.5; during: chargePct=chargePct-4;		
Empty entry: sentPower=0;		
	Discharge	
	[isCharging]	[chargePct<100]
	Charge	Empty
	[isCharging]	
	Charge	

- 3 In the **Debug** tab, click **Breakpoints List**.

The second row in the table represents the breakpoint on the Charge state. To trigger the breakpoint when the chargePct data is more than 100, double-click the **Condition** column and enter chargePct>100.

Enabled	Source	Source Type	Condition	Hits
☒	sfGetStartedBatteryTable/.../...	Transition Valid		0
☒	sfGetStartedBatteryTable/.../...	State During	chargePct>100	0

- 4 To simulate the model, in the **Simulation** tab, click **Run**. The simulation pauses when the Charge state is active and chargePct is greater than 100.
- 5 Toggle the Manual Switch block. In the **Simulation** tab, click **Continue**. The simulation pauses when the transition from Charge to Discharge is valid.

In the next step of the tutorial, you prevent the battery charge from exceeding 100% by adding child states to the Charge state.

See Also

Related Examples

- “Monitor State Activity Through Active State Data”
- “Set Breakpoints to Debug Charts”
- “Inspect and Modify Data and Messages While Debugging”

Create Parent and Child Operating Modes in State Transition Tables

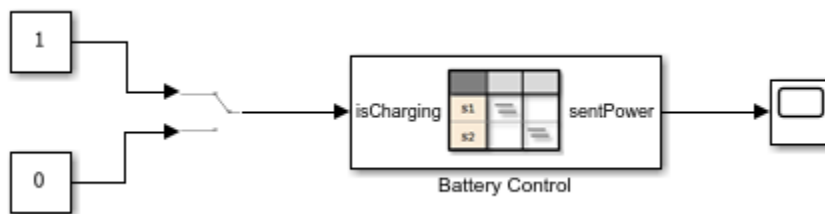
You can create more complex logic in a state transition table by creating child states that can activate only when their parent state is active. In the **States** column, states are indented under their parent states.

Child states have their own default transitions. When the parent state becomes active, the child state connected to the default transition also becomes active. Then, the table transitions between the child states. When the parent state deactivates, the child states also deactivate.

In this example, you use child states to model the variable charging rates in a rechargeable battery system.

Open Model

The `sfGetStartedBatteryTable` model represents the control logic for a rechargeable battery system. To build the model, follow the instructions in the previous steps of the tutorial.



The State Transition Table block **Battery Control** contains the control logic for the system. To determine whether the battery charges or discharges, the table receives a Manual Switch block as an input.

To view the table, double-click the State Transition Table block.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
Charge entry: sentPower=0; during: chargePct=chargePct+3;	[~isCharging]	
	Discharge	
Discharge entry: sentPower=3.5; during: chargePct=chargePct-4;	[isCharging]	[chargePct<=4]
	Charge	Empty
Empty entry: sentPower=0;	[isCharging]	
	Charge	

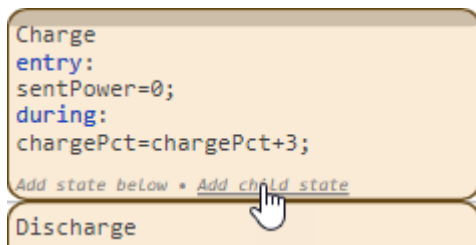
The states Charge, Discharge, and Empty represent the operating modes of the battery system. The table uses these data:

- **isCharging**: An input that determines whether the battery should charge or discharge.
- **chargePct**: A local data that represents the charge level of the battery as a percent of total charge.
- **sentPower**: An output that determines how many watts of power the battery sends at any given time.

Add Child States to Model Hierarchy

Currently, the battery system continues charging when full. To prevent the battery from charging beyond full, add states representing fast, intermediate, and full charging modes.

- 1 Point to the Charge state and click **Add child state**. Name the child state FastCharge.



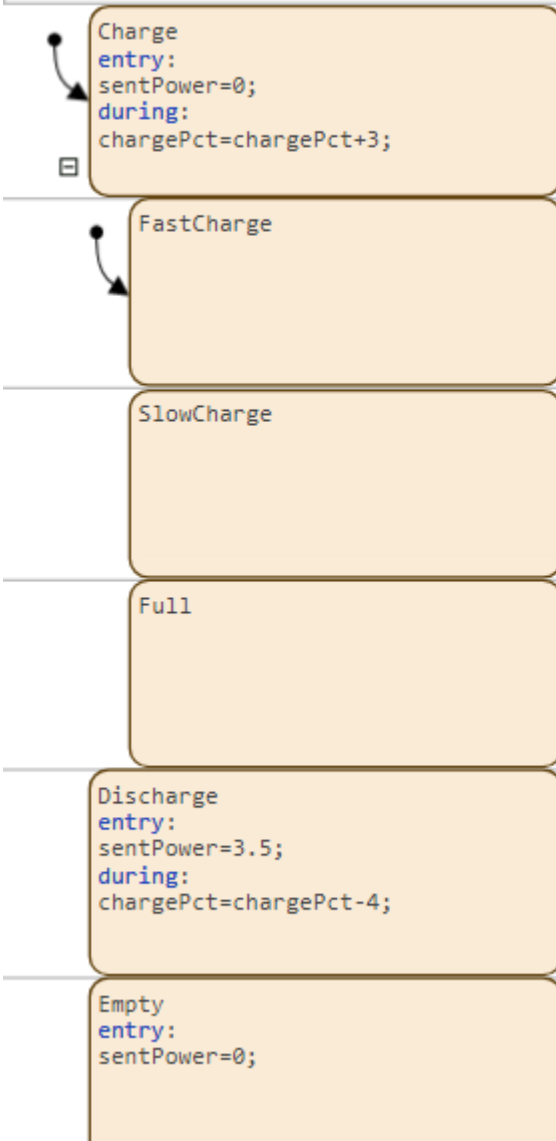
- 2 Add two additional child states named SlowCharge and Full.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
Charge entry: sentPower=0; during: chargePct=chargePct+3; ⊖	[~isCharging]	
	Discharge	
FastCharge		
SlowCharge		
Full		
Discharge entry: sentPower=3.5; during: chargePct=chargePct-4;	[isCharging]	[chargePct<=4]
	Charge	Empty
Empty entry: sentPower=0;	[isCharging]	
	Charge	

Note To hide child states, on the left side of the parent state, click the Hide child states button . To show hidden states, click the Show child states button .

Add transitions that allow the table to move between the charging modes.

- 1 Create a transition from FastCharge to SlowCharge that executes when the battery charge is greater than 80%.
- 2 Create a transition from SlowCharge to Full that executes when the battery charge is greater than or equal to 100%.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 <pre> stateDiagram-v2 [*] --> Charge Charge --> Discharge : [~isCharging] Discharge --> Charge : [isCharging] Charge --> FastCharge : [chargePct>80] FastCharge --> SlowCharge : [chargePct>=100] SlowCharge --> Full : [chargePct>=100] Full --> Empty : [isCharging] Empty --> Charge : [isCharging] </pre> Charge entry: sentPower=0; during: chargePct=chargePct+3; FastCharge SlowCharge Full Discharge entry: sentPower=3.5; during: chargePct=chargePct-4; Empty entry: sentPower=0;	[~isCharging] Discharge [chargePct>80] SlowCharge [chargePct>=100] Full [isCharging] Charge [isCharging] Charge	[chargePct<=4] Empty

Add state actions that increase the battery charge.

- 1 In the Charge state, remove the existing during actions.
- 2 In the FastCharge state, add a during action that increases the battery charge by 4%. In the SlowCharge state, add a during action that increases the battery charge by 1%.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 Charge entry: sentPower=0;	[~isCharging]	
	Discharge	
FastCharge during: chargePct=chargePct+4;	[chargePct>80]	
	SlowCharge	
SlowCharge during: chargePct=chargePct+1;	[chargePct>=100]	
	Full	
Full		
Discharge entry: sentPower=3.5;	[isCharging]	[chargePct<=4]
	Charge	Empty
Empty entry: sentPower=0;	[isCharging]	
	Charge	

Simulate Model

Observe how the table transitions between child states.

- 1 To slow the pace of the simulation, in the **Simulation** tab, click **Run > Simulation Pacing**. Then, select **Enable pacing to slow down simulation**.
- 2 In the top model, confirm that the Manual Switch block connects to the Constant block with a value of 1.

- 3 In the **Simulation** tab, click **Run** to simulate the model. In the first step, the **Charge** and **FastCharge** states both activate. As charge increases, the table transitions to child states **SlowCharge** and **Full**.

The battery model now stops charging when the battery charge is 100%. In the next step of the tutorial, you add a non-rechargeable emergency battery that maintains essential functions if the main battery runs out of power. To model the simultaneous operation of the batteries, you use parallel states.

See Also

Related Examples

- “Use State Hierarchy to Design Multilevel State Complexity”
- “Move Between Levels of Hierarchy by Using Supertransitions”
- “Execution of a Stateflow Chart”

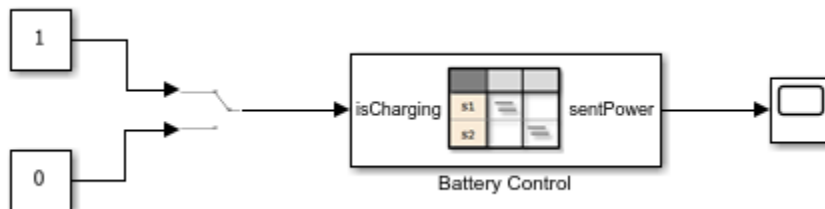
Execute States in Parallel in State Transition Tables

You can simultaneously activate every state in the same level of hierarchy of a state transition table by using parallel decomposition. When a table or state that uses parallel decomposition becomes active, every direct child of the table or state also becomes active. The children, which are called *parallel states*, appear in the table with a dashed border.

In this step of the tutorial, you use parallel states to add a non-rechargeable emergency battery to the battery control system. If the main battery runs out of power, the backup battery maintains its essential functions.

Open Model

The `sfGetStartedBatteryParallelTable` model represents the control logic for a rechargeable battery system.



The State Transition Table block `Battery Control` contains the control logic for the system. To determine whether the battery charges or discharges, the table receives a Manual Switch block as an input.

To view the table, double-click the State Transition Table block. To see additional states, scroll up or down.

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
 Main		
 Charge entry: sentPower=0;	[~isCharging]	
 FastCharge during: mainChrg=mainChrg+3;	[mainChrg>80]	
SlowCharge during: mainChrg=mainChrg+1;	[mainChrg>=100]	
Full		
Discharge entry: sentPower=3.5; during: mainChrg=mainChrg-4	[isCharging]	[mainChrg<=4]
Empty entry: sentPower=0;	[isCharging]	
Emergency		
 Standby		
Discharge		

The table has two non-parallel states that represent the main and emergency batteries. Each battery has nested child states that represent operating modes for charging, discharging, running out of power, or standing by. Additionally, the batteries use these data:

- **isCharging**: An input that determines whether the battery should charge or discharge.
- **mainChrg**: A local data that represents the charge level of the main battery as a percent of total charge.
- **emerChrg**: A local data that represents the charge level of the emergency battery as a percent of total charge.
- **sentPower**: An output that determines how many watts of power the battery system sends at any given time.

For details on the operation of the main battery, see the previous steps in the tutorial.

Enable Parallel Decomposition

Enable parallel execution for child states by right-clicking the parent component and selecting **Decomposition > Parallel**.

Because the **Main** and **Emergency** states do not have a parent state, the table is the parent. To enable parallel decomposition for the table, in the **Modeling** tab, select **Decomposition > Parallel (AND)**. The borders of the two top-level states become dashed, which indicates they are parallel.

Note To hide child states, on the left side of the parent state, click the Hide child states button . To show hidden states, click the Show child states button .

STATES	TRANSITIONS	
	IF	ELSE-IF(2)
Main		
Charge entry: sentPower=0;	[~isCharging]	
Discharge entry: sentPower=3.5; during: mainChrg=mainChrg-4	Discharge ▼	▼
Empty entry: sentPower=0;	[isCharging]	[mainChrg<=4]
	Charge ▼	Empty ▼
Emergency		
Standby		
Discharge	Discharge ▼	▼
	Standby ▼	▼

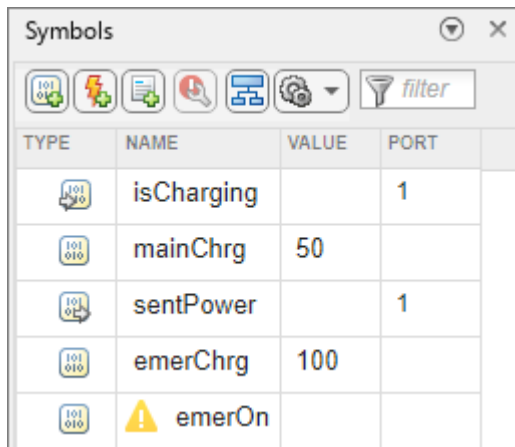
In state transition tables, states that use parallel decomposition do not support transitions. In this table, the transition columns for parallel states Main and Emergency are dimmed.

Coordinate Behavior Across Parallel States

When using parallel states, you can trigger behavior one state based on the activity of another. For example, in the battery system, the emergency battery switches on only if the main battery becomes empty. In state transition tables, you can coordinate behavior across parallel states by assigning a value to local data in one state, and creating a transition condition that uses the local data in another state.

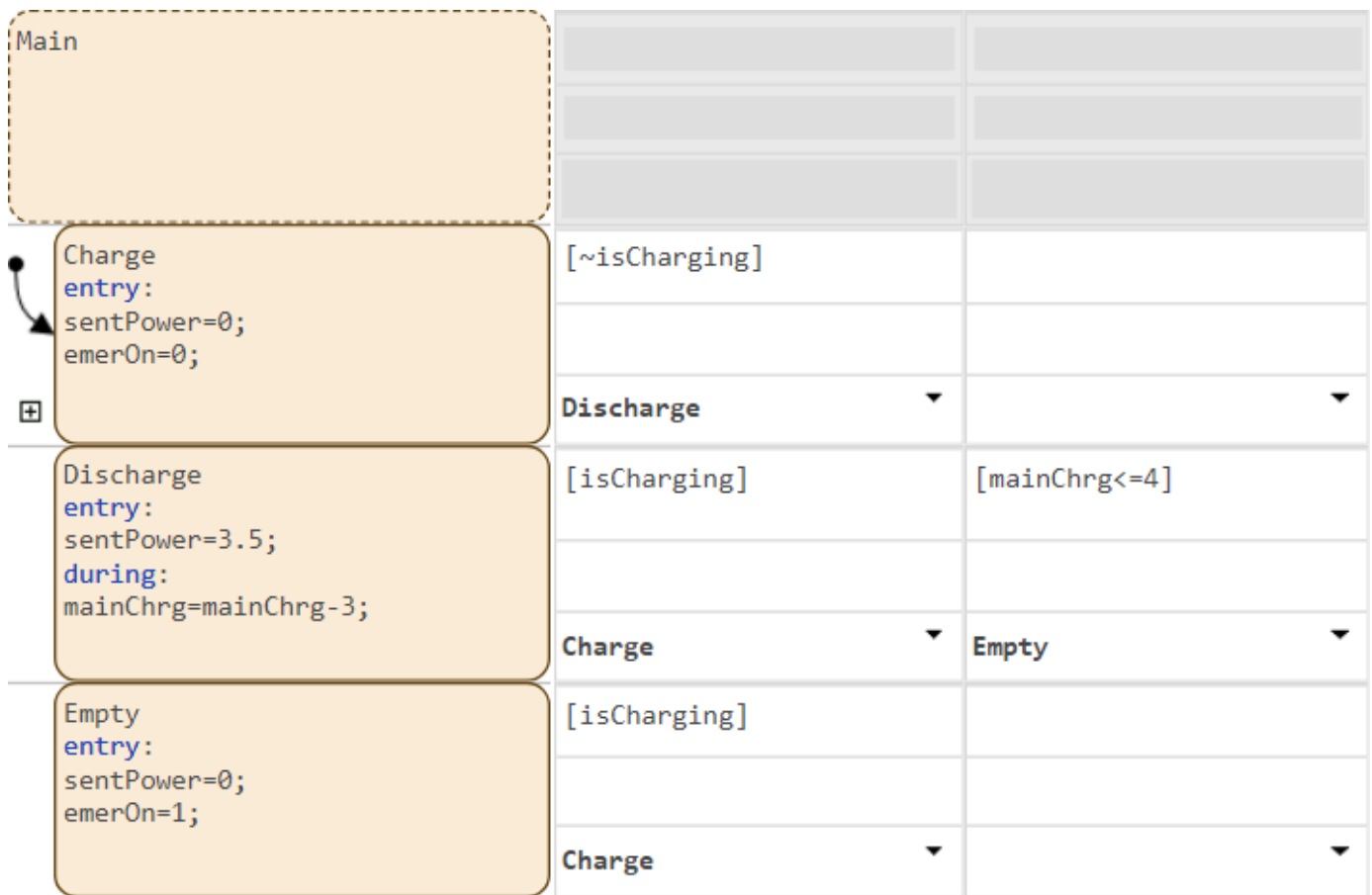
Create local data that allows the main battery to activate or deactivate the emergency battery.

- 1 In the **Symbols** pane, click the Create Data button . Name the data emerOn.

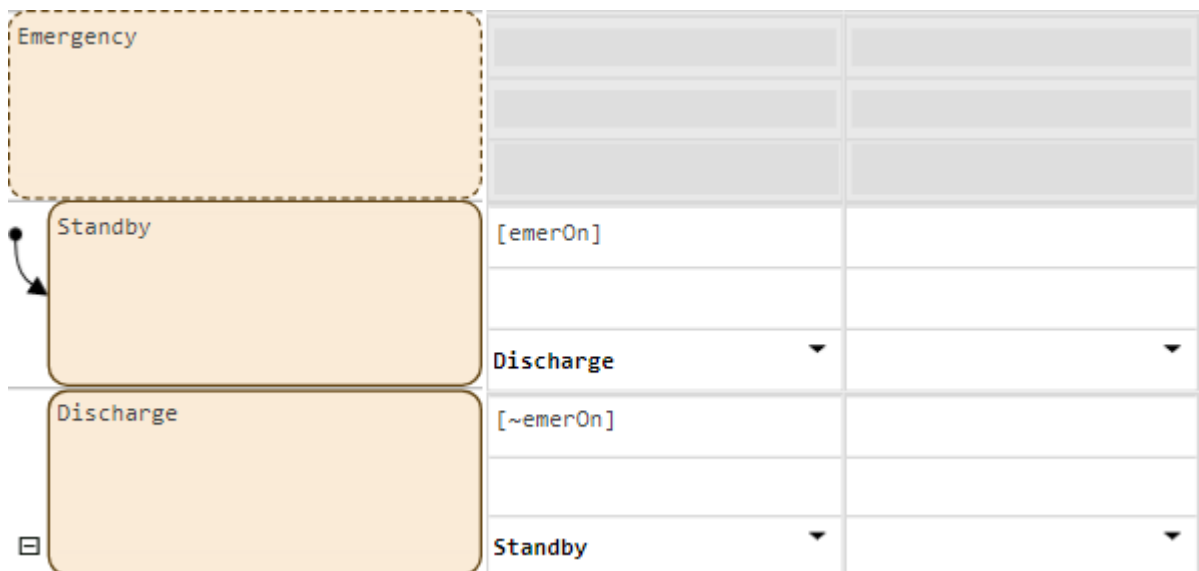


TYPE	NAME	VALUE	PORT
	isCharging		1
	mainChrg	50	
	sentPower		1
	emerChrg	100	
	emerOn		

- 2 In the Main state, in the Charge child state, in the entry action, set emerOn to 0. In the Empty child state, in the entry action, set emerOn to 1.



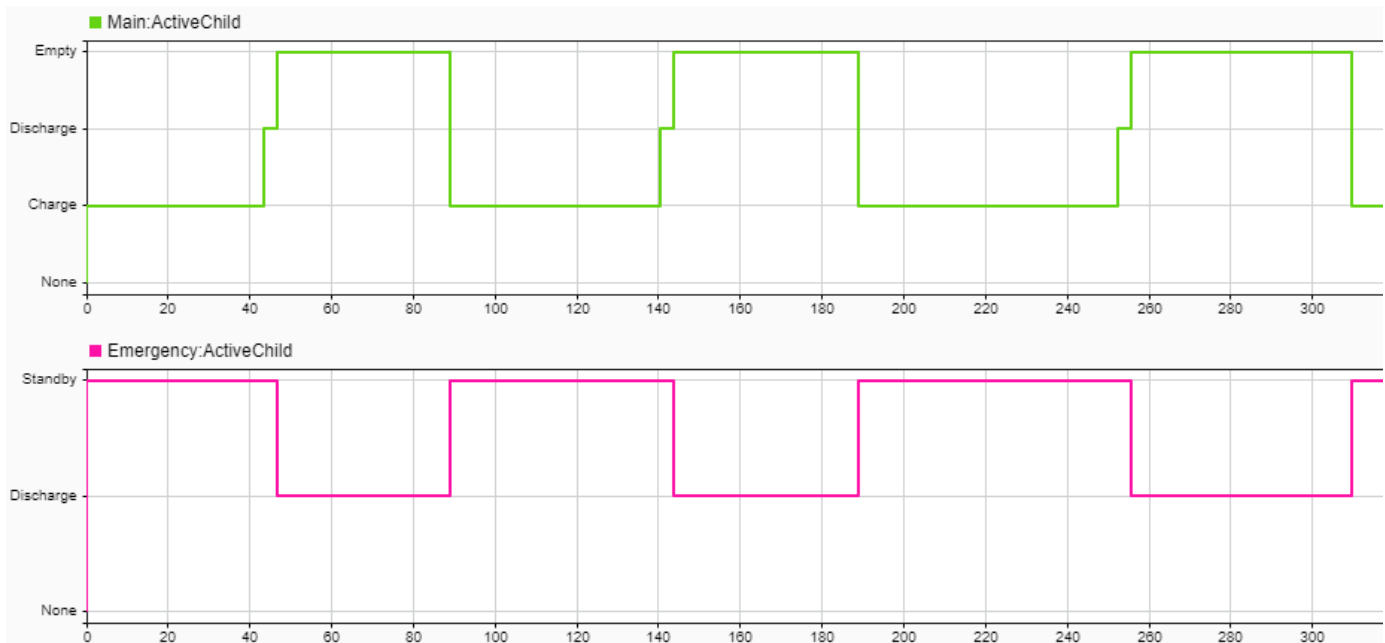
- 3 In Emergency, in the Standby state, add a transition that activates the Discharge state if emerOn is true.
- 4 In Emergency, in the Discharge state, add a transition that activates the Standby state if emerOn is false.



Simulate Model

Simulate the model.

- 1 Return to the top-level Simulink model. The Manual Switch block changes the operating mode of the main battery. When the value is 1, the battery charges. When the value is 0, the battery discharges. Toggle the Manual Switch block so that it connects to the Constant block with a value of 1.
- 2 To simulate the model, in the **Simulation** tab, click **Run**.
- 3 Double-click the Manual Switch block. After a few seconds, double-click the Manual Switch block again. Repeat this process several times.
- 4 To stop the simulation, in the **Simulation** tab, click **Stop**.
- 5 To open the Simulation Data Inspector, in the **Simulation** tab, click **Data Inspector**.
- 6 To show the active child of each parent state, in the **Inspect** tab, select **Main:ActiveChild** and **Emergency:ActiveChild**.



When the main battery is empty, the emergency battery discharges. When the main battery leaves the Empty state and begins charging, the emergency battery moves to standby.

See Also

Related Examples

- “Use Events to Execute Charts”
- “Define Exclusive and Parallel Modes by Using State Decomposition”
- “Monitor State Activity Through Active State Data”

